



Aspectos Técnicos de CNNs em PyTorch

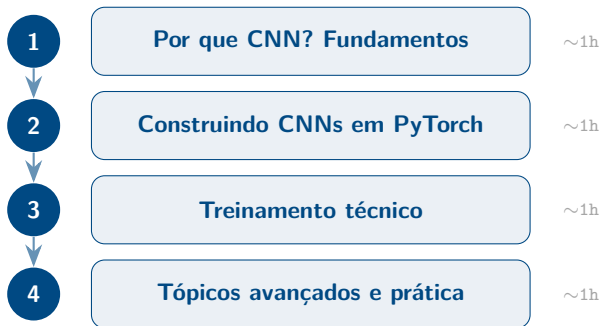
Aprendizagem de Máquina — CDIA

Prof. Dr. Rooney R. A. Coelho

Graduação em Ciência de Dados e Inteligência Artificial — CDIA

2026

Roteiro da aula (~4h)



Material

 Slides PDF  Notebooks de apoio  Goodfellow et al., *Deep Learning*, cap. 9

O que você já sabe (e não vamos repetir)

✓ `nn.Module` + `forward`

✓ `DataLoader`, `Dataset`

✓ TensorBoard, schedulers

✓ Loop de treino + Adam/SGD

✓ `CrossEntropyLoss`, GPU

✓ AlexNet pré-treinada

Hoje a aula assume tudo isso. Vamos direto ao que faz uma rede ser **convolucional**: como funciona a operação de convolução, como montar arquiteturas, como treinar com qualidade e como usar redes pré-treinadas de forma sistemática.

Objetivos da aula

Ao final, você deve ser capaz de:

- **Explicar** por que MLPs não são ideais para imagens.
- Calcular **shape de saída** de qualquer **Conv2d/MaxPool2d**.
- Montar uma CNN **do zero** (LeNet, depois algo no nível de ResNet simples) em PyTorch.
- Aplicar **BatchNorm**, **Dropout**, **data augmentation** e **schedulers** corretamente.
- Diferenciar **feature extraction** de **fine-tuning**.
- Visualizar filtros, *feature maps* e **Grad-CAM**.
- Usar **mixed precision** e boas práticas de reprodutibilidade.

Promessa

No fim de hoje, você não vai mais “copiar uma CNN da internet”. Vai entender cada linha, cada hiperparâmetro, e saber exatamente *o que ajustar* quando o treino não convergir.



Por que CNN? Fundamentos da convolução

O problema motivador

Queremos classificar imagens. Uma opção é *achatar* a imagem e jogar num MLP que já sabemos construir:

```
class MLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        self.fc1 = nn.Linear(3*224*224, 1024) # ???
        self.fc2 = nn.Linear(1024, 10)
    def forward(self, x):
        return self.fc2(F.relu(self.fc1(self.flatten(x))))
```

Quantos parâmetros?

Só a **fc1**: $3 \cdot 224 \cdot 224 \cdot 1024 \approx 154$ milhões pesos. Para uma única camada. Em uma rede pequena. Vamos entender por que isso é um problema.

Onde os 154 milhões vão morar

Imagem (3, 224, 224) achatada é um vetor de comprimento $3 \cdot 224^2 = 150.528$. Cada neurônio da próxima camada precisa de um peso para cada entrada:

Camada	Saída	Pesos (sem bias)
Linear 150528 $\rightarrow$ 1024	1024	154.140.672
Linear 1024 $\rightarrow$ 512	512	524.288
Linear 512 $\rightarrow$ 256	256	131.072
Linear 256 $\rightarrow$ 1000	1000	256.000
Total		~155 milhões

Para referência

A ResNet-50 tem ~25 milhões de parâmetros *para a rede inteira* – e bate em precisão qualquer MLP nessa tarefa. CNN é **melhor com menos parâmetros**, não apesar disso.

Três problemas do MLP em imagens

Explosão de parâmetros

154M numa só camada
p/ imagem 224x224

Sem *prior* espacial

Pixel vizinho
tratado igual
a pixel longe

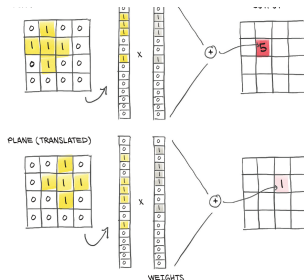
Sem invariância translacional

Gato canto sup. e
gato canto inf.
são “imagens diferentes”

CNN ataca os três de uma vez:

- Compartilha o mesmo filtro por toda a imagem $\Rightarrow$ menos parâmetros.
- Operadores locais (kernel pequeno) $\Rightarrow$ usa estrutura espacial.
- Mesmo filtro detecta o padrão em qualquer posição $\Rightarrow$ invariância.

MLP perde a invariância translacional



O *mesmo* avião em duas posições ativa **pesos diferentes** de uma camada **Linear**. A rede fully-connected não *enxerga* que é o mesmo padrão deslocado.

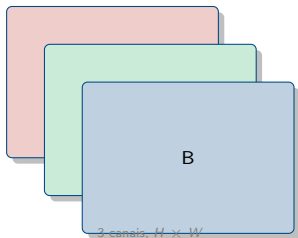
O argumento decisivo: parâmetros explodem

Em uma imagem 1024×1024 RGB ($\approx 3.1\text{M}$ pixels), um MLP com só 1024 unidades ocultas na primeira camada teria **~ 3 bilhões** de parâmetros. Em float32, isso é **~ 12 GB de RAM só nos pesos da primeira camada** – antes mesmo de calcular qualquer gradiente. “Just not going to fit on most present-day GPUs.”

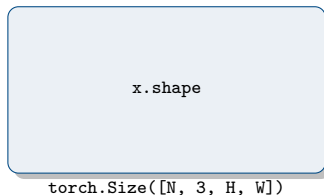
Fontes: Stevens et al., *DLPT*, fig. 7.16 (p. 190) e argumento da p. 189.

Imagem como tensor $[N, C, H, W]$

Imagem RGB



Tensor 4D



PyTorch usa o formato $[N, C, H, W]$: N = batch, C = canais, H = altura, W = largura. Toda `Conv2d` espera essa ordem. Imagem `PIL` ou `numpy` costuma vir como $[H, W, C]$ – por isso `ToTensor()` faz a permutação automática.

Normalização de imagens

Após `ToTensor()`, pixels viram `float` em $[0, 1]$. Normalizar (média 0, desvio 1) acelera muito o treino:

```
# Estatísticas do ImageNet -- usadas com TODA rede pre-treinada
transforms.Normalize(
    mean=[0.485, 0.456, 0.406],
    std =[0.229, 0.224, 0.225]
)
```

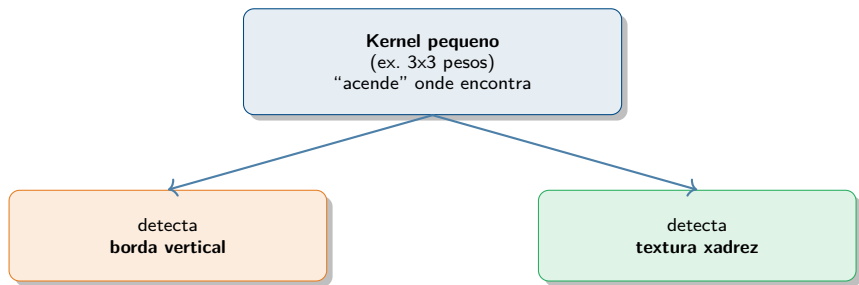
Por que importa

Por canal: $x' = (x - \mu_c) / \sigma_c$. Garante que cada canal entra na rede em uma escala parecida, ativações ficam centradas, e o gradiente não explode em uma direção. Se você treinou seu modelo com normalização, **na inferência precisa normalizar igual**.

Armadilha

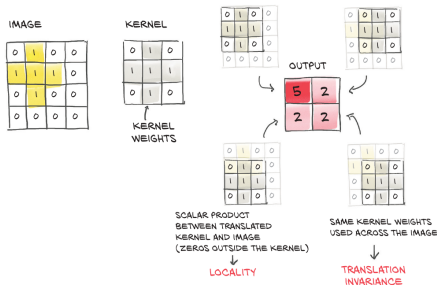
Esqueceu de normalizar na inferência? Precisão despenca sem dar erro. É o erro silencioso mais comum em produção.

Intuição: filtro = detector de padrão local



O mesmo kernel **passa por toda a imagem**. Onde o padrão do kernel “casa” com a região da imagem, o produto interno fica alto. Onde não casa, fica baixo. Resultado: um *mapa de ativação* (*feature map*) que “acende” onde o padrão existe.

Convolução = produto escalar local repetido



Para cada posição do output, fazemos o **produto escalar** entre o kernel e a janela local. Duas propriedades-chave: **localidade** (só vizinhança) e **invariância translacional** (*mesmos pesos em toda a imagem*).

Detalhe técnico (vale saber)

Em matemática, convolução envolve um *flip* do kernel. Em deep learning – e no PyTorch – não fazemos esse flip: o que chamamos de `nn.Conv2d` é tecnicamente uma **correlação cruzada discreta**. Como o kernel é aprendido, a rede “aprende o kernel já flipado” se for o caso. Você vai encontrar o termo *cross-correlation* em papers.

Fonte: Stevens, Antiga, Viehmann – *Deep Learning with PyTorch*, fig. 8.1 e nota 1, p. 195–196.

Antes do DL: kernels feitos à mão (Sobel)



Kernels **Sobel** (detectores de borda) eram a base da visão computacional clássica. Aplicados manualmente:

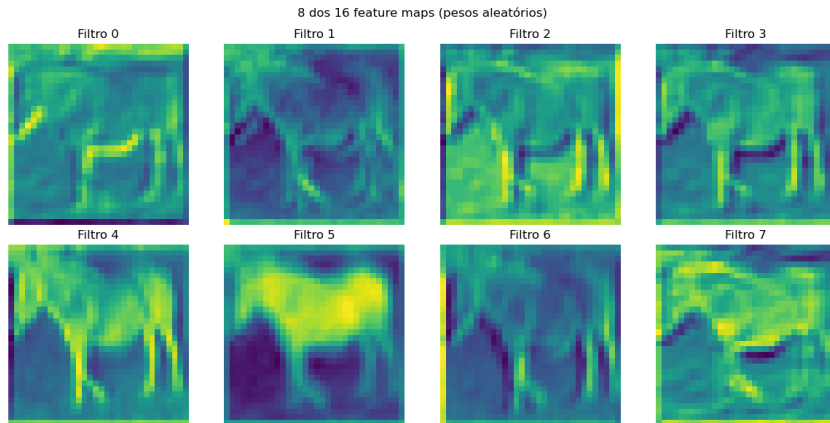
$$\mathbf{K}_x = [[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]], \mathbf{K}_y = \mathbf{K}_x^T.$$

O que mudou com DL

Você não escolhe mais o kernel. A rede aprende pelos gradientes qual padrão detectar para resolver a tarefa. Sobel é o exemplo canônico do que cada [Conv2d](#) aprende sozinho na primeira camada.

Saída real do [01_fundamentos_convolucao.ipynb](#).

Feature maps com kernel *aleatório*



Mesma imagem, passada por uma `Conv2d(3, 16, 3)` com pesos *aleatórios*. Cada filtro produz um *feature map* – e mesmo não-treinados, já destacam *algumas* estruturas (filtro 5 “acendeu” o cavalo, filtro 1 destacou o fundo).

O treino *melhora* esses filtros

Estes são kernels randômicos. Depois do treino com backprop, eles convergem para detectores úteis (bordas, texturas, cores...) – exatamente como nos slides anteriores. (Saída real do `01_fundamentos_convolucao.ipynb`.)

Convolução 2D – passo 1

Entrada 5x5

1	2	3	0	1
0	1	2	3	1
1	0	1	2	0
2	1	0	1	2
1	2	1	0	1

Kernel 3x3

1	0	-1
1	0	-1
1	0	-1

Saída 3x3

-2	?	?
?	?	?
?	?	?

Conta: $1 \cdot 1 + 2 \cdot 0 + 3 \cdot (-1) + 0 \cdot 1 + 1 \cdot 0 + 2 \cdot (-1) + 1 \cdot 1 + 0 \cdot 0 + 1 \cdot (-1) = -2$

O kernel começa no canto superior esquerdo. Multiplica elemento a elemento, soma tudo, escreve *um* número na saída.

Convolução 2D – passo 2 (desliza 1)

Entrada 5x5

1	2	3	0	1
0	1	2	3	1
1	0	1	2	0
2	1	0	1	2
1	2	1	0	1

Kernel 3x3

1	0	-1
1	0	-1
1	0	-1

Saída 3x3

-2	-1	?
?	?	?
?	?	?

Próxima janela: kernel desliza 1 pixel à direita (*stride*=1).

Repetimos a multiplicação + soma para cada posição da janela na entrada. Cada posição gera um pixel de saída.

Convolução 2D – resultado completo

Entrada 5x5

1	2	3	0	1
0	1	2	3	1
1	0	1	2	0
2	1	0	1	2
1	2	1	0	1

Kernel

1	0	-1
1	0	-1
1	0	-1

*

=

Saída 3x3

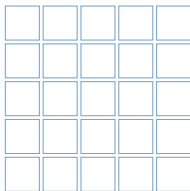
-2	-1	1
-3	1	2
1	2	-1

Entrada 5×5 + kernel 3×3 sem padding, stride 1 $\Rightarrow$ saída 3×3 . **Saída é menor que a entrada** (perdemos as bordas). Solução: *padding*.

Nota técnica: o que chamamos de "convolução" em DL é tecnicamente uma correlação cruzada (sem flip do kernel). Não muda a prática.

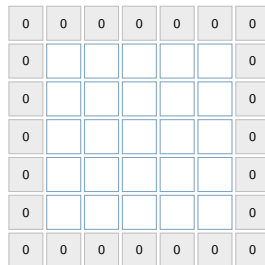
Padding – preservando o tamanho

Valid ($P = 0$)



saída 3x3

Same ($P = 1$)



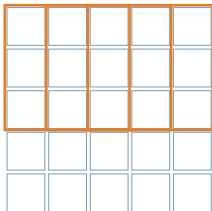
saída 5x5

Padding = adicionar zeros (ou outro valor) ao redor da entrada. Com $P = 1$ em kernel 3×3 , saída fica do mesmo tamanho que a entrada. Em PyTorch:

```
nn.Conv2d(in_c, out_c, kernel_size=3, padding=1) # mantém H, W  
nn.Conv2d(in_c, out_c, kernel_size=3, padding=0) # encolhe
```

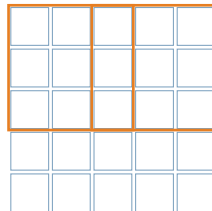
Stride – passos maiores reduzem o mapa

stride=1



$$\text{saída } \frac{5-3}{1} + 1 = 3$$

stride=2



$$\text{saída } \frac{5-3}{2} + 1 = 2$$

Stride = quantos pixels o kernel desloca a cada passo. **stride=2** reduz a resolução espacial pela metade. É uma forma de *downsampling* sem usar pooling.

```
nn.Conv2d(64, 128, kernel_size=3, padding=1, stride=2) # H, W -> H/2, W/2
```

Stride 2 vs MaxPool 2x2 – qual escolher?

Os dois reduzem H, W pela metade. Como decidir?

```
# Op. A: conv stride 2 -- downsample com pesos aprendíveis
self.down = nn.Conv2d(64, 128, kernel_size=3, stride=2, padding=1)

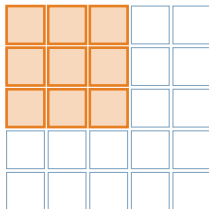
# Op. B: conv stride 1 + maxpool -- downsample com max fixo
self.conv = nn.Conv2d(64, 128, kernel_size=3, padding=1)
self.pool = nn.MaxPool2d(2)
```

Regra prática

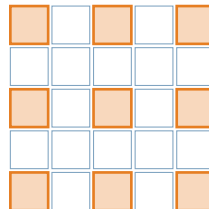
- **Conv stride 2**: usado em ResNet, VGG moderna. Mais flexível, mais parâmetros aprendidos.
- **MaxPool 2x2**: usado em LeNet, AlexNet clássica. Sem parâmetros, mais rápido.
- Em prática, conv stride 2 é ligeiramente melhor.

Dilation – janela esparsa, campo receptivo maior

dilation=1 (normal)



dilation=2



Dilation = espaço entre os elementos do kernel. Com $\text{dilation}=2$, um kernel 3×3 “olha” uma região 5×5 usando os mesmos 9 pesos. Aumenta *receptive field* sem aumentar parâmetros.

Onde aparece

Segmentação semântica (DeepLab) e modelos de áudio (WaveNet) usam dilation forte para capturar contexto largo.

Fórmula do tamanho de saída (*decore*)

$$H_{\text{out}} = \left\lfloor \frac{H_{\text{in}} + 2P - D(K - 1) - 1}{S} \right\rfloor + 1$$

Variáveis

- H_{in} = altura de entrada (mesma fórmula para largura).
- K = `kernel_size`.
- P = `padding` (zeros adicionados de cada lado).
- S = `stride` (passo).
- D = `dilation` (1 = normal).

Caso comum (kernel 3, padding 1, stride 1, dilation 1)

$H_{\text{out}} = \lfloor (H + 2 - 2 - 1)/1 \rfloor + 1 = H$. **Tamanho preservado.** É por isso que “`kernel=3, padding=1`” é o padrão universal.

Exercícios rápidos da fórmula

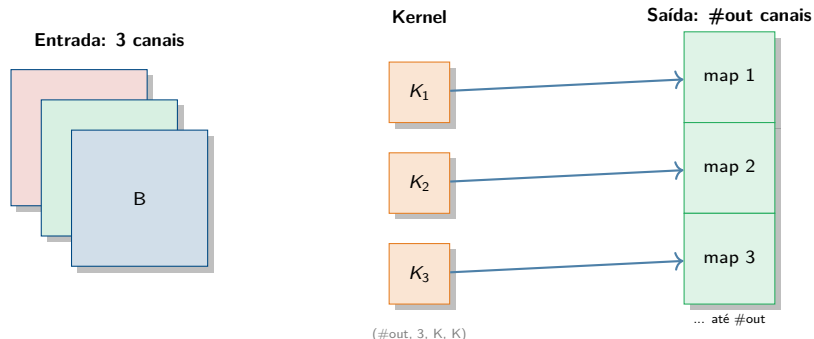
Dilation = 1 em todos os exemplos.

Caso	H_{in}	K	P	S	H_{out}	Comentario
1	28	5	0	1	24	LeNet primeira conv
2	28	3	1	1	28	"same padding" clássico
3	224	7	3	2	112	ResNet conv1
4	224	3	1	2	112	alternativa para halve
5	32	3	1	1	32	padrão CIFAR-10
6	32	2	0	2	16	MaxPool 2x2 stride 2

Verifique o caso 3 você mesmo

$$H_{out} = \lfloor (224 + 2 \cdot 3 - 1 \cdot (7 - 1) - 1) / 2 \rfloor + 1 = \lfloor (224 + 6 - 6 - 1) / 2 \rfloor + 1 = \lfloor 223 / 2 \rfloor + 1 = 111 + 1 = 112.$$

Conv com múltiplos canais (entrada e saída)



Conv2d com `in_channels=3`, `out_channels=16` tem **16 kernels**, cada um com formato $3 \times K \times K$. Cada kernel "vê" os 3 canais e produz *um* feature map. Total de parâmetros: $16 \cdot 3 \cdot K \cdot K + 16$ (bias).

Group convolution e depthwise separable

`groups` divide os canais de entrada em grupos independentes. Cada filtro vê *apenas* seu grupo.

```
# Normal: cada kernel ve todos os canais de entrada
nn.Conv2d(64, 128, kernel_size=3, padding=1, groups=1) # padrao

# Group conv (ResNeXt usa): metade canais para cada grupo
nn.Conv2d(64, 128, kernel_size=3, padding=1, groups=2)

# Depthwise: 1 filtro POR canal (MobileNet)
nn.Conv2d(64, 64, kernel_size=3, padding=1, groups=64) # 64 kernels 3x3

# Depthwise separable = depthwise + pointwise (1x1)
self.dw = nn.Conv2d(64, 64, kernel_size=3, padding=1, groups=64)
self.pw = nn.Conv2d(64, 128, kernel_size=1) # combina canais
```

Por que importa

Depthwise separable usa $\sim 10\times$ menos parâmetros que conv normal. É o motor do MobileNet, EfficientNet e outras redes feitas para celular.

Compartilhamento de parâmetros

O mesmo kernel é aplicado em **toda a imagem**. Compare:

```
# Linear "equivalente" para imagem 32x32 colorida
nn.Linear(3*32*32, 16*32*32) # 50_331_648 parametros

# Conv equivalente
nn.Conv2d(3, 16, kernel_size=3, padding=1) # 432 + 16 = 448
```

Mais de 100.000 vezes menos parâmetros

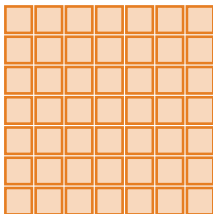
É *melhor* generalização: como o mesmo filtro detecta o padrão em qualquer lugar, ele “aprende” com todos os exemplos de cada padrão na imagem. Isso é **equivariância translacional**: `conv(translate(x)) = translate(conv(x))`.

Não confundir

Equivariância $\neq$ invariância. A invariância (saída indiferente à posição) vêm do *pooling* e do tipo de *head* no final.

Campo receptivo (*receptive field*)

Camada 0 (entrada)



Camada 2 (conv 3x3 stride 1)



3x3 saídas
(cada uma com r.f. 5x5)

Camada 3 (conv 3x3 stride 1)



1 saída c/ r.f. 7x7

Empilhando 3 convs 3×3 , o pixel final “vê” uma região 7×7 da entrada. Fórmula recursiva:

$$RF_l = RF_{l-1} + (K_l - 1) \prod_{i < l} S_i.$$

Camadas mais profundas vêem padrões maiores: bordas $\rightarrow$ texturas $\rightarrow$ partes $\rightarrow$ objetos.

Receptive field cresce com a profundidade

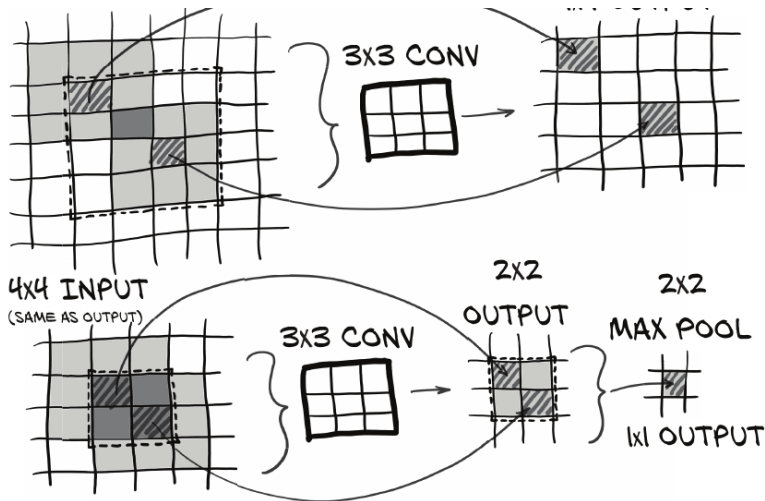


Figure 13.5 The convolutional architecture of a `LunaModel` block, consisting of two 3×3 convolutions followed by a max pool. The final pixel has a 6×6 receptive field.

Duas convs 3×3 em sequência: cada pixel do *output* final “vê” uma região 6×6 da entrada. Com mais camadas e pooling, a janela visível cresce rápido sem aumentar o tamanho do kernel.

Por que o *pooling* muda o jogo

Crescimento geométrico do receptive field

Sem pooling, RF cresce *aritméticamente*: L camadas com $K=3$ dão $RF = 2L + 1$. Com pooling intercalado, RF cresce *geometricamente*: cada Pool2 dobra o stride efetivo das camadas seguintes. Por isso uma stack como Conv3 → Pool2 → Conv3 já tem $RF = 8 \times 8$ – e não precisamos de kernels enormes.

Dois $3 \times 3 <$ um 5×5

Dois Conv3 empilhados têm $RF = 5 \times 5$ e usam menos parâmetros que um único Conv5: $2 \cdot 9 = 18$ pesos por canal contra 25. E ainda dão *duas* não-linearidades em vez de uma. Por isso VGG, ResNet e quase todas as CNNs modernas usam 3×3 por padrão.

Fontes: Stevens et al., *DLPT*, p. 205 e p. 292.

nn.Conv2d – a assinatura completa

```
nn.Conv2d(  
    in_channels, # canais da entrada (3 RGB, ou C anterior)  
    out_channels, # n. de kernels = n. de feature maps na saída  
    kernel_size, # int ou (kH, kW)  
    stride=1, # int ou (sH, sW)  
    padding=0, # int, tupla, ou 'same'/'valid' (PyTorch >= 1.9)  
    dilation=1,  
    groups=1, # depthwise se groups == in_channels  
    bias=True, # False se próxima camada é BatchNorm  
    padding_mode='zeros' # 'reflect', 'replicate', 'circular'  
)
```

Dicas práticas

- `kernel_size=3, padding=1`: combo padrão, mantém H, W .
- `bias=False` quando vem `BatchNorm` em seguida (BN tem seu próprio *shift*).
- `groups>1` ativa *group convolution* (depthwise/MobileNet).

Convolução 1×1 – por que existe

Parece inútil ($K = 1$ não olha vizinhos). Mas *atua nos canais*: **combina linearmente os canais em cada pixel**.

```
# Reduzir canais (bottleneck em ResNet, Inception)
self.reduce = nn.Conv2d(256, 64, kernel_size=1) # 256 -> 64 canais

# Expandir canais
self.expand = nn.Conv2d(64, 256, kernel_size=1)
```

Três usos comuns

- **Reduzir canais** antes de uma conv 3×3 cara (Inception, ResNet bottleneck).
- Adicionar **não-linearidade** sem mexer em H, W (segue uma ReLU).
- Atuar como uma camada **fully connected pixel a pixel** – equivalente.

Inicialização de pesos

Pesos são inicializados aleatoriamente. A escolha da distribuição importa para o gradiente não explodir/sumir

	Esquema	Fórmula (var.)	Quando
nas primeiras iterações.	Xavier (Glorot)	$\sigma^2 = \frac{2}{n_{in} + n_{out}}$	tanh, sigmoid
	He (Kaiming)	$\sigma^2 = \frac{2}{n_{in}}$	ReLU
	Uniforme	$\mathcal{U}(-a, a)$ com a apropriado	legado

Em PyTorch, padrão do Conv2d

Inicialização uniforme de Kaiming já é o *default* do `nn.Conv2d` (versions modernas). Para customizar:

```
def init_weights(m):
    if isinstance(m, nn.Conv2d):
        nn.init.kaiming_normal_(m.weight, mode='fan_out', nonlinearity='relu')
    if m.bias is not None:
        nn.init.zeros_(m.bias)
model.apply(init_weights)
```

Pooling – reduzir dimensão, ganhar invariância

Entrada 4x4

1	3	2	4
5	2	1	3
2	4	6	1
3	1	2	5

MaxPool 2x2

5	4
4	6

AvgPool 2x2

2.75	2.5
2.5	3.5

MaxPool pega o máximo da janela; **AvgPool** pega a média. Reduz H , W e dá um pouco de invariância a pequenos deslocamentos. Sem parâmetros aprendidos.

Max pooling em detalhe

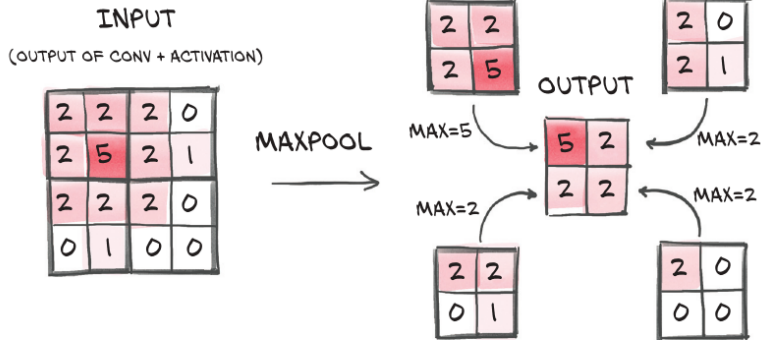


Figure 8.7 Max pooling in detail

Cada janela 2×2 do input vira UM pixel no output – o **máximo** dos 4. O resultado: meia resolução, ruído local atenuado, e os “picos de ativação” preservados.

Fonte: Stevens, Antiga, Viehmann – *Deep Learning with PyTorch*, fig. 8.7, p. 203.

Pooling em PyTorch

```
nn.MaxPool2d(kernel_size=2, stride=2) #  $H, W \rightarrow H/2, W/2$ 
nn.AvgPool2d(kernel_size=2, stride=2) # mesma forma, calcula media

# Adaptive: voce diz o tamanho da SAIDA, ele descobre o stride
nn.AdaptiveAvgPool2d(output_size=(1, 1)) # qualquer  $H, W \rightarrow 1 \times 1$ 
nn.AdaptiveAvgPool2d(output_size=(7, 7)) # qualquer  $H, W \rightarrow 7 \times 7$ 

# Global Average Pooling -- chave em ResNets modernas
x = nn.AdaptiveAvgPool2d(1)(x) #  $(N, C, H, W) \rightarrow (N, C, 1, 1)$ 
x = torch.flatten(x, 1) #  $\rightarrow (N, C)$ 
```

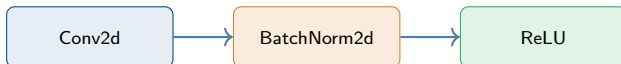
Por que Adaptive é tão útil

Sua rede aceita imagens de *qualquer tamanho*: o `AdaptiveAvgPool2d(1)` normaliza para um vetor fixo independente da resolução da entrada. Foi o que substituiu o `"view(-1, 16*5*5)"` mágico das CNNs antigas.

BatchNorm2d – normalização no meio da rede

Para cada canal c , no batch atual:

$$\hat{x}_c = \frac{x_c - \mu_c}{\sqrt{\sigma_c^2 + \epsilon}}, \quad y_c = \gamma_c \hat{x}_c + \beta_c.$$



Benefícios

- Estabiliza distribuição das ativações camada a camada.
- Permite **learning rates maiores** (treino até 10x mais rápido).
- Atua como regularizador suave.
- Quase sempre *melhora* a acurácia final.

Detalhe importante

Em `eval()`, BN usa estatísticas *móveis* (acumuladas no treino), não do batch atual. Por isso esquecer `model.eval()` na inferência quebra a saída.

BatchNorm2d em código

```
self.conv = nn.Conv2d(64, 128, kernel_size=3, padding=1, bias=False)
self.bn = nn.BatchNorm2d(128) # mesmo num. de canais da saída da conv
self.act = nn.ReLU(inplace=True)

# forward:
x = self.act(self.bn(self.conv(x)))
```

Hiperparâmetros internos

- `num_features`: o número de canais da camada anterior.
- `eps`: ϵ numérico, padrão 10^{-5} .
- `momentum`: peso da média móvel (padrão 0.1).
- `affine=True`: aprende γ, β .
- `track_running_stats=True`: mantém média móvel para inferência.

Quando `bias=False` no `Conv2d`

BN já tem β aprendido. O `bias` do `Conv` é redundante e somar dois `bias` não traz benefício.

Dropout em conv vs dropout em FC

`nn.Dropout(p)`

Zera *neurônios individuais* (cada ativação escalar com prob p). Usado após camadas `Linear`.

```
nn.Linear(512, 10),  
nn.Dropout(p=0.5),
```

`nn.Dropout2d(p)`

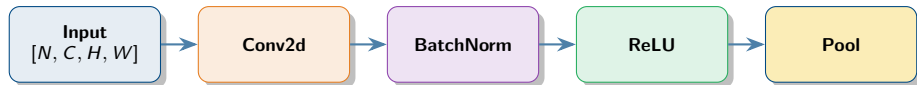
Zera *canais inteiros* (todo um feature map). Mais adequado em CNN porque pixels vizinhos são muito correlacionados.

```
nn.Conv2d(64, 128, 3, padding=1),  
nn.Dropout2d(p=0.1),
```

Cuidado

Dropout fica **ativo só em treino**. `model.eval()` desliga automaticamente. Se você esquecer o `eval()`, o resultado varia a cada inferência.

Anatomia de um bloco convolucional típico



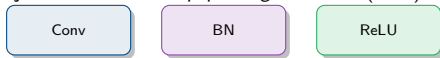
Padrão recorrente em qualquer CNN moderna: **Conv** → **BN** → **ReLU** → **Pool**. O bloco aparece repetido várias vezes; a cada repetição H , W encolhem e o número de canais cresce.

Por que essa ordem

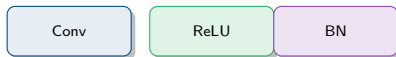
Conv produz *logits* sem não-linearidade → BN normaliza esses logits → ReLU corta os negativos → Pool reduz resolução. Inverter a ordem (ex. BN antes da Conv) costuma piorar o treino.

Ordem “Conv-BN-ReLU” vs “Conv-ReLU-BN”

Há dois jeitos de ordenar. O *paper original* da BN (2015) deixou ambíguo; a literatura convergiu na primeira



padrão atual



pouco usado

ordem.

Por que Conv-BN-ReLU: BN normaliza a entrada da ReLU (média 0, desv. 1) – isso evita que muitos neurônios fiquem em modo “morto” (sempre 0). Em prática, melhora um pouquinho a precisão e dá treino mais estável.

Funções de ativação em CNNs

Por décadas todas usavam *sigmoid* ou *tanh*, que saturam e desaparecem o gradiente. A ascensão das CNNs

	Ativação	Fórmula	Observação
profundas veio junto da ReLU.	ReLU	$\max(0, x)$	padrão em CNNs modernas
	LeakyReLU	$\max(0.01x, x)$	evita "ReLU morto"
	ELU	x se $x > 0$, $\alpha(e^x - 1)$ senão	ativações médias ≈ 0
	GELU	$x \cdot \Phi(x)$	padrão em Transformers
	SiLU/Swish	$x\sigma(x)$	EfficientNet

Recomendação prática

Para CNN de classificação em CDIA: **ReLU** é o suficiente, **rápido** e amplamente testado.
`nn.ReLU(inplace=True)` economiza um pouco de memória.

MLP vs CNN: parâmetros para classificar CIFAR-10

Imagem $32 \times 32 \times 3$, 10 classes.

Arquitetura	#parâmetros	Acurácia típica
MLP $3072 \rightarrow 1024 \rightarrow 256 \rightarrow 10$	3.409.930	~50%
LeNet adaptada	~62.000	~70%
CNN com 5 blocos + BN	~300.000	~85%
ResNet-18 do zero	~11.000.000	~92%

O mais impressionante

A LeNet adaptada tem **55x menos parâmetros que o MLP** e atinge **20pp a mais de acurácia**. Mais parâmetros sem o viés indutivo certo é *pior*, não melhor.

Resumo do Bloco 1

Conv2d
 K, P, S, D

Pooling
Max, Avg, Adaptive

Compartilhamento
+equivariância

Fórmula
$$\lfloor (H + 2P - D(K - 1) - 1) / S \rfloor + 1$$

BatchNorm2d
 $\gamma \hat{x} + \beta$

Campo receptivo
cresce em profundidade

Mensagem central: convolução = filtro compartilhado + *prior* de localidade. Tudo o que vamos ver adiante é como empilhar e treinar esses blocos.



Construindo CNNs em PyTorch

nn.Sequential vs nn.Module customizado

Sequential (linear)

```
model = nn.Sequential(  
    nn.Conv2d(3, 16, 3, padding=1),  
    nn.BatchNorm2d(16),  
    nn.ReLU(inplace=True),  
    nn.MaxPool2d(2),  
    nn.Flatten(),  
    nn.Linear(16*16*16, 10),  
)
```

Quando usar: fluxo é estritamente em linha reta, sem ramificações nem *skip*.

nn.Module

```
class CNN(nn.Module):  
    def __init__(self):  
        super().__init__()  
        self.c1 = nn.Conv2d(3,16,3,padding=1)  
        self.fc = nn.Linear(16*16*16, 10)  
    def forward(self, x):  
        x = F.relu(self.c1(x))  
        x = F.max_pool2d(x, 2)  
        x = torch.flatten(x, 1)  
        return self.fc(x)
```

Quando usar: precisa de *skip connections*, múltiplas saídas, ou lógica no `forward`.

Padrão: criar um ConvBlock reutilizável

```
class ConvBlock(nn.Module):  
    """Conv -> BN -> ReLU (-> opcional MaxPool)."""  
    def __init__(self, in_c, out_c, kernel=3, pad=1, pool=False):  
        super().__init__()  
        self.conv = nn.Conv2d(in_c, out_c, kernel, padding=pad, bias=False)  
        self.bn = nn.BatchNorm2d(out_c)  
        self.act = nn.ReLU(inplace=True)  
        self.pool = nn.MaxPool2d(2) if pool else nn.Identity()  
  
    def forward(self, x):  
        return self.pool(self.act(self.bn(self.conv(x))))
```

Pra que serve

Você escreve o bloco *uma* vez e empilha: `ConvBlock(3,32)`, `ConvBlock(32,64,pool=True)`, etc. Código fica curto, testado e fácil de mudar (só um lugar para adicionar dropout, mudar ativação, etc).

LeNet-5 (Yann LeCun, 1998)

A primeira CNN que viralizou. Foi treinada para reconhecer dígitos manuscritos em cheques bancários. **2 conv**



Princípios que continuam válidos hoje

Empilhar Conv + Pool + Conv + Pool, depois cabeça totalmente conectada. **H,W diminui, canais aumentam.** Tudo veio dessa arquitetura.

LeNet-5 em PyTorch

```
class LeNet5(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 6, kernel_size=5), # 32x32 -> 28x28
            nn.ReLU(inplace=True),
            nn.AvgPool2d(2), # 28 -> 14
            nn.Conv2d(6, 16, kernel_size=5), # 14 -> 10
            nn.ReLU(inplace=True),
            nn.AvgPool2d(2), # 10 -> 5
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(16*5*5, 120), nn.ReLU(inplace=True),
            nn.Linear(120, 84), nn.ReLU(inplace=True),
            nn.Linear(84, num_classes),
        )
    def forward(self, x):
        return self.classifier(self.features(x))
```

Calculando o shape antes da fc1

Como saber que vai dar “16*5*5” antes da `Linear`?

Camada	K	P	S	Saída $H \times W$
input	-	-	-	32x32
Conv2d(1,6,5)	5	0	1	28x28
AvgPool2d(2)	2	0	2	14x14
Conv2d(6,16,5)	5	0	1	10x10
AvgPool2d(2)	2	0	2	5x5
Flatten	-	-	-	$16 \cdot 5 \cdot 5 = 400$

Truque prático

Se você não quer calcular, faça:

```
with torch.no_grad():
    dummy = torch.zeros(1, 1, 32, 32)
    out_features = self.features(dummy).flatten(1).shape[1]
self.classifier = nn.Linear(out_features, 10)
```

Treinando a LeNet-5 em MNIST

```
from torchvision import datasets, transforms

tf = transforms.Compose([transforms.Pad(2), transforms.ToTensor()])
ds_tr = datasets.MNIST('data', train=True, download=True, transform=tf)
ds_te = datasets.MNIST('data', train=False, download=True, transform=tf)
loader_tr = DataLoader(ds_tr, batch_size=128, shuffle=True, num_workers=2)
loader_te = DataLoader(ds_te, batch_size=256, num_workers=2)

modelo = LeNet5().to(device)
criterio = nn.CrossEntropyLoss()
opt = torch.optim.Adam(modelo.parameters(), lr=1e-3)

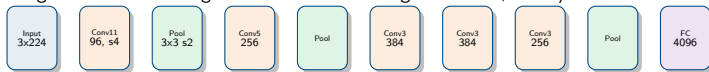
for epoca in range(10):
    modelo.train()
    for x, y in loader_tr:
        x, y = x.to(device), y.to(device)
        opt.zero_grad(); loss = criterio(modelo(x), y); loss.backward(); opt.step()

# Acuracia tipica apos 5 epocas: > 99%
```

Trinta segundos de treino e ela ultrapassa 99%. **MNIST é fácil demais para o estado da arte de hoje** – existe até como desafio “LeNet sem treino”.

AlexNet (Krizhevsky et al., 2012)

O divisor de águas. Venceu o ImageNet de 2012 com margem enorme; começou o *boom* da deep learning.



Novidades em relação à LeNet

ReLU (em vez de tanh), **Dropout** nas FC, **Data augmentation**, treino com **2 GPUs** em paralelo, imagens grandes (224x224). ~60 milhões de parâmetros.

AlexNet em PyTorch (esqueleto)

```
class AlexNet(nn.Module):
    def __init__(self, num_classes=1000):
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(3, 64, 11, stride=4, padding=2), nn.ReLU(True),
            nn.MaxPool2d(3, stride=2),
            nn.Conv2d(64, 192, 5, padding=2), nn.ReLU(True),
            nn.MaxPool2d(3, stride=2),
            nn.Conv2d(192, 384, 3, padding=1), nn.ReLU(True),
            nn.Conv2d(384, 256, 3, padding=1), nn.ReLU(True),
            nn.Conv2d(256, 256, 3, padding=1), nn.ReLU(True),
            nn.MaxPool2d(3, stride=2),
        )
        self.avgpool = nn.AdaptiveAvgPool2d((6, 6))
        self.classifier = nn.Sequential(
            nn.Dropout(0.5), nn.Linear(256*6*6, 4096), nn.ReLU(True),
            nn.Dropout(0.5), nn.Linear(4096, 4096), nn.ReLU(True),
            nn.Linear(4096, num_classes),
        )
    def forward(self, x):
        return self.classifier(torch.flatten(self.avgpool(self.features(x)), 1))
```

Você não precisa decorar. `torchvision.models.alexnet()` faz tudo. Mas entender o esqueleto explica o que veio depois.

VGG-16 (Simonyan & Zisserman, 2014)

Tudo com kernel 3x3 e profundidade extrema. Blocos de 2–3 convs seguidos de maxpool. Foi a primeira a passar dos 90% top-5 no ImageNet com receita simples.

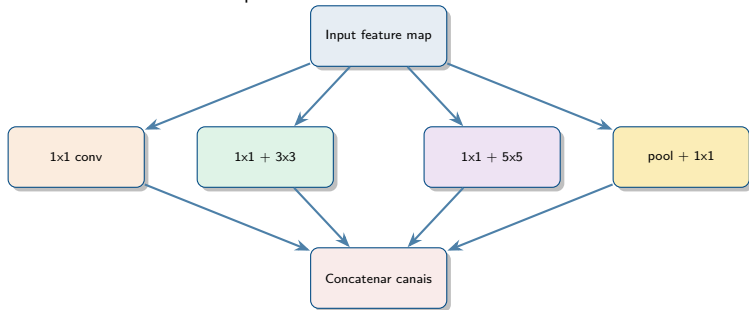


Insight da VGG

Duas convs 3×3 empilhadas têm **mesmo campo receptivo** de uma conv 5×5 , com *menos parâmetros* ($2 \cdot 9 = 18$ vs 25) e *mais não-linearidade*. Hoje, todas as CNNs usam 3×3 por padrão graças a essa observação.

GoogLeNet (Inception) – 2014

Em vez de escolher um kernel fixo, **aplica vários em paralelo** e concatena os resultados. Cada caminho captura padrões em uma escala diferente.



O truque: convolução 1x1

1×1 é uma combinação linear dos canais em cada pixel. Serve para **reduzir número de canais** antes do $3 \times 3 / 5 \times 5$, economizando muita computação.

O problema das redes muito profundas

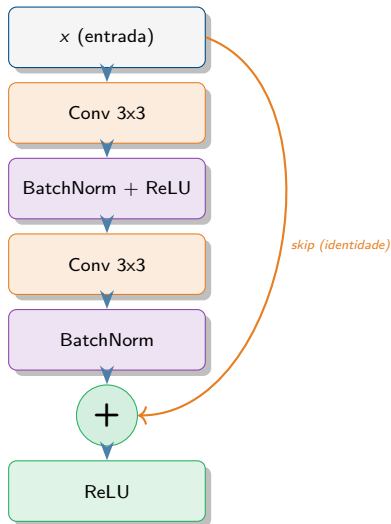
Mais profundo *deveria* ser melhor. Mas não era: ao passar de ~ 20 camadas, o erro de *treino aumentava*. Não era overfitting – era um problema de **otimização**.



Vanishing gradient

A regra da cadeia multiplica derivadas de ~ 0.5 por camada. Em 30 camadas, $0.5^{30} \approx 10^{-9}$. As camadas iniciais mal aprendem.

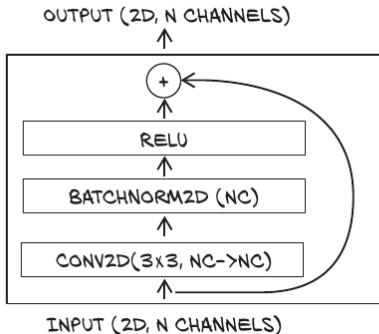
ResNet (He et al., 2015) – bloco residual



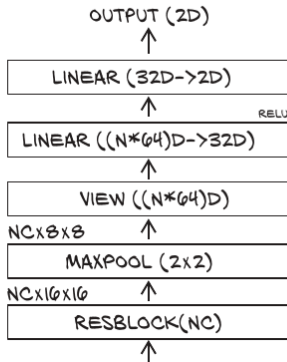
Saída = $\mathcal{F}(x) + x$. Se a rede não precisa de mais nada, basta aprender $\mathcal{F} \approx 0$ – bem mais fácil que aprender a identidade num bloco normal. **Permitiu redes de 152 camadas** (e mais).

Bloco residual em diagrama

RESBLOCK



NETRESDEEP



ResBlock (esq.): a entrada *salta* sobre Conv + BatchNorm + ReLU e é somada ao final. **NetResDeep** (dir.): rede profunda empilhando vários ResBlocks antes do classifier.

Fonte: Stevens, Antiga, Viehmann – *Deep Learning with PyTorch*, fig. 8.12, p. 226.

Bloco residual em PyTorch

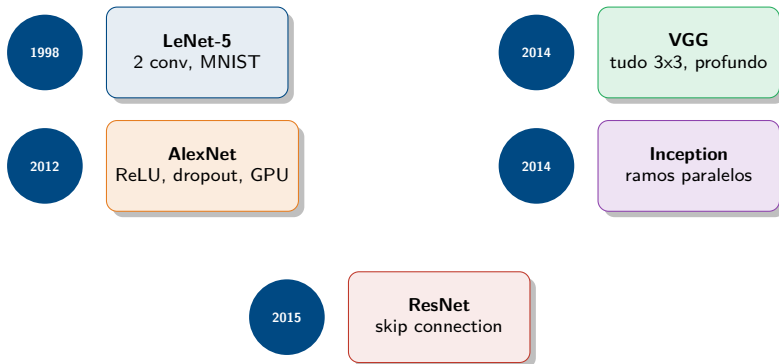
```
class ResidualBlock(nn.Module):
    def __init__(self, c):
        super().__init__()
        self.conv1 = nn.Conv2d(c, c, 3, padding=1, bias=False)
        self.bn1 = nn.BatchNorm2d(c)
        self.conv2 = nn.Conv2d(c, c, 3, padding=1, bias=False)
        self.bn2 = nn.BatchNorm2d(c)

    def forward(self, x):
        identity = x # <- guarda o input
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        out = out + identity # <- skip connection
        return F.relu(out)
```

Detalhe

Quando o número de canais muda, a `identity` precisa de uma projeção 1x1 para bater os shapes. Isso aparece nas “downsample” das ResNets do [torchvision](#).

Linha do tempo das CNNs clássicas



Padrão geral: cada geração quebra um limite da anterior. ReLU + dropout (AlexNet) → kernels pequenos (VGG) → múltiplas escalas (Inception) → profundidade viável (ResNet) → *tradeoffs* compute/precisão (EfficientNet, em 2019) → atenção (ViT, em 2020).

Datasets clássicos – escolha por tamanho

Dataset	Imagens	Classes	Resolução
MNIST	70k	10	28×28 cinza
Fashion MNIST	70k	10	28×28 cinza
CIFAR-10	60k	10	32×32 RGB
CIFAR-100	60k	100	32×32 RGB
Tiny-ImageNet	110k	200	64×64 RGB
ImageNet	1.2M+	1000	$\sim 224 \times 224$ RGB

Para estudar/prototipar

Comece com CIFAR-10 (treino em minutos numa GPU mediana, números não triviais para superar). MNIST é muito fácil hoje. ImageNet é o “mundo real” – mas precisa de muita compute.

Inspecionando shapes da sua CNN

Pacote `torchinfo` mostra a saída de cada camada e o total de parâmetros:

```
from torchinfo import summary

modelo = LeNet5()
summary(modelo, input_size=(1, 1, 32, 32))
```

```
=====
Layer (type:depth-idx) Output Shape Param #
=====
LeNet5 [1, 10] --
\---Sequential: 1-1 [1, 16, 5, 5] --
    \---Conv2d: 2-1 [1, 6, 28, 28] 156
    \---ReLU: 2-2 [1, 6, 28, 28] --
    \---AvgPool2d: 2-3 [1, 6, 14, 14] --
    \---Conv2d: 2-4 [1, 16, 10, 10] 2,416
    \---ReLU: 2-5 [1, 16, 10, 10] --
    \---AvgPool2d: 2-6 [1, 16, 5, 5] --
\---Sequential: 1-2 [1, 10] --
    \---Flatten: 2-7 [1, 400] --
    \---Linear: 2-8 [1, 120] 48,120
    \---Linear: 2-9 [1, 84] 10,164
    \---Linear: 2-10 [1, 10] 850
=====
Total params: 61,706
```

Contando parâmetros sem o torchinfo

```
# Total
total = sum(p.numel() for p in modelo.parameters())

# Apenas os treinaveis
treinaveis = sum(p.numel() for p in modelo.parameters() if p.requires_grad)

print(f"Total: {total:,}")
print(f"Treinaveis: {treinaveis:,}")

# Por camada
for nome, p in modelo.named_parameters():
    print(f"{nome:40s} {tuple(p.shape):20s} {p.numel():,}")
```

Por que diferenciar treinaveis

Em *transfer learning* com congelamento, `requires_grad=False` exclui parâmetros do otimizador. Total continua igual, mas treinaveis caem drasticamente – e é isso que de fato vai para o otimizador.

Sanity check: passar um tensor zero pela rede

Antes de treinar, valide que os shapes batem em todo o `forward`:

```
modelo = MinhaCNN()
modelo.eval()

# Cria entrada fake do tamanho esperado
dummy = torch.zeros(2, 3, 32, 32) # batch 2, 3 canais, 32x32

with torch.no_grad():
    out = modelo(dummy)

print("Saida:", out.shape) # esperado: (2, num_classes)
```

Por que sempre fazer

Detecta cedo: shape errado entre conv e fc, dimensão trocada, esqueceu `flatten`, etc. Custa 2 segundos e poupa hora de debug.

Truque para fc automática

Você pode usar `nn.LazyLinear(num_classes)` – ela descobre o número de features de entrada no primeiro `forward`.

CNN do zero para CIFAR-10 – parte 1

Padrão moderno (cap. 11 do livro): **tail** (adapta a entrada), **backbone** (extrai features) e **head** (decide a classe). Mesmo vocabulário que veremos em transfer learning.

```
class CIFAR10CNN(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        # TAIL: normaliza a entrada (BN sobre canais RGB)
        self.tail = nn.BatchNorm2d(3)

        # BACKBONE: extrai features hierarquicas
        self.backbone = nn.Sequential(
            ConvBlock(3, 32),
            ConvBlock(32, 64, pool=True), # 32 -> 16
            ConvBlock(64, 128),
            ConvBlock(128, 128, pool=True), # 16 -> 8
            ConvBlock(128, 256),
            ConvBlock(256, 256, pool=True), # 8 -> 4
        )

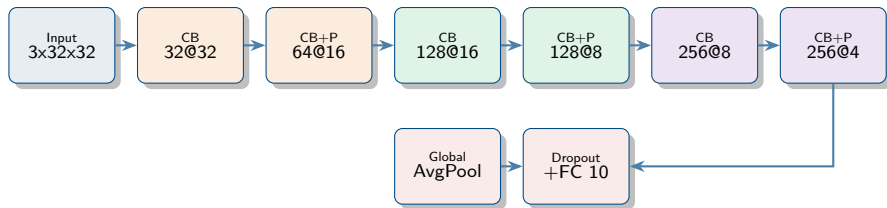
        # HEAD: pool global + classifier
        self.head = nn.Sequential(
            nn.AdaptiveAvgPool2d(1), # 4x4 -> 1x1
            nn.Flatten(),
            nn.Dropout(0.3),
            nn.Linear(256, num_classes),
        )

    def forward(self, x):
        return self.head(self.backbone(self.tail(x)))
```

Por que esse vocabulário

Em *transfer learning* (bloco 4), substituir a **head** é a operação-padrão; congelar o **backbone** e treinar só a **head** é exatamente o que faremos no bolsolula. Aprender o nome aqui poupa confusão depois. (Stevens et al., p. 290–293.)

CIFAR-10 CNN – diagrama da arquitetura

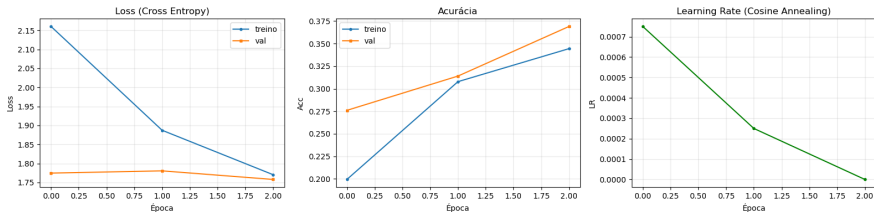


Padrão: blocos duplos. O 2º de cada par tem pool, dobrando os canais e dividindo H , W por 2. Termina com Global Average Pooling – elimina necessidade de calcular " $X * H * W$ ".

Total de parâmetros

~1.2M, já o suficiente para passar de 88% no CIFAR-10 com data augmentation.

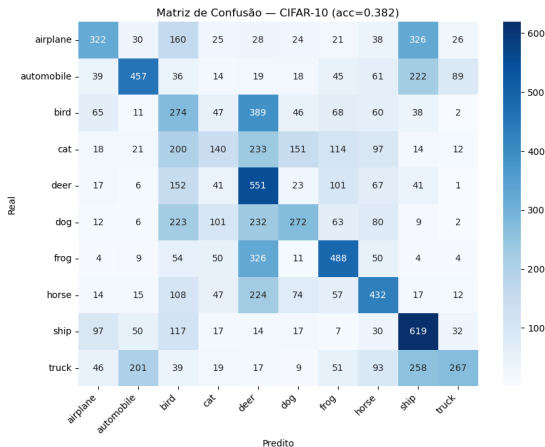
Treino real: nossa CNN no CIFAR-10 (notebook 02)



Treino *de demonstração*: 5000 imagens, 3 épocas, AdamW + CosineAnnealingLR. Loss desce monotonicamente, acurácia sobe. Note como **val acc > train acc** no começo – efeito da *augmentation* no treino (mais difícil que val).

Saída real do `02_cnn_cifar10_completa.ipynb`.

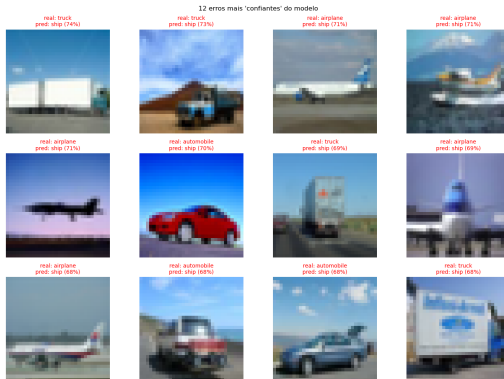
Matriz de confusão – onde o modelo erra



Diagonal forte = acertos. Erros mais comuns: **airplane↔ship**, **deer/bird** (animais voadores ou caminchantes em paisagem), **cat/dog**.

Saída real do [02_cnn_cifar10_completa.ipynb](#).

“Erros confiantes” – modelo aprendeu o atalho



12 imagens em que o modelo errou com 68–74% de confiança. Todas **previstas como “ship”**. O comum entre elas? Fundo azul claro / céu / horizonte.

Shortcut features

O modelo aprendeu “céu azul $\Rightarrow$ ship” em vez da forma do barco. Biais do dataset que só aparece olhando os erros. Saída real do [02_cnn_cifar10_completa.ipynb](#).

Debugando shapes no forward

Quando você não sabe se o shape está certo:

```
class CNN(nn.Module):
    def __init__(self):
        super().__init__()
        self.c1 = nn.Conv2d(3, 16, 3, padding=1)

    def forward(self, x):
        print("antes:", x.shape)
        x = self.c1(x)
        print("apos c1:", x.shape)
        return x

modelo(torch.zeros(2, 3, 32, 32))
# antes: torch.Size([2, 3, 32, 32])
# apos c1: torch.Size([2, 16, 32, 32])
```

Limpar os prints depois!

Esses prints rodam a cada batch, em todas as épocas. Apague ou comente quando terminar de debugar – você não quer 40000 prints por época.

print(model) – estrutura textual

```
>>> print(LeNet5())
LeNet5(
  (features): Sequential(
    (0): Conv2d(1, 6, kernel_size=(5, 5), stride=(1, 1))
    (1): ReLU(inplace=True)
    (2): AvgPool2d(kernel_size=2, stride=2, padding=0)
    (3): Conv2d(6, 16, kernel_size=(5, 5), stride=(1, 1))
    (4): ReLU(inplace=True)
    (5): AvgPool2d(kernel_size=2, stride=2, padding=0)
  )
  (classifier): Sequential(
    (0): Flatten(start_dim=1, end_dim=-1)
    (1): Linear(in_features=400, out_features=120, bias=True)
    (2): ReLU(inplace=True)
    (3): Linear(in_features=120, out_features=84, bias=True)
    (4): ReLU(inplace=True)
    (5): Linear(in_features=84, out_features=10, bias=True)
  )
)
```

Ferramenta rápida de inspeção – não mostra shapes intermediários (só a configuração). Para shapes, use `torchinfo.summary` ou `print` no `forward`.

Resumo do Bloco 2

Sequential vs Module
linear vs custom

LeNet → ResNet
evolução 1998-2015

ConvBlock
Conv+BN+ReLU(+Pool)

torchinfo
shapes e #params

AdaptiveAvgPool2d(1)
fim mais elegante que view

Próximo bloco: como treinar com qualidade – data augmentation, schedulers, TensorBoard, checkpoints, reprodutibilidade.



Treinamento técnico

Antes de tudo: dividir os dados

Nada de “trein e testa no mesmo set”. Sempre três partições:

```
from torch.utils.data import random_split

ds = datasets.ImageFolder('dados/todos', transform=transform_treino)

# Tamanhos: 70 / 15 / 15
n_total = len(ds)
n_tr = int(0.70 * n_total)
n_vl = int(0.15 * n_total)
n_te = n_total - n_tr - n_vl

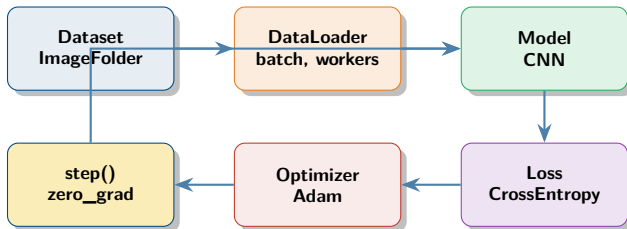
g = torch.Generator().manual_seed(42) # reprodutivel
ds_tr, ds_vl, ds_te = random_split(ds, [n_tr, n_vl, n_te], generator=g)

# IMPORTANTE: val e teste DEVEM usar transform SEM augmentation
ds_vl.dataset.transform = transform_val
ds_te.dataset.transform = transform_val
```

Armadilha do random_split

Como ele só muda os *índices*, todos compartilham o mesmo `dataset.transform`. **Se você mudar para *augmentation*, vai aumentar também na validação.** O jeito limpo: criar três Datasets separados (um por transform) ou usar Subset com proxy.

Pipeline de treino para imagens



Você já fez esse loop com MLP. Hoje a diferença está no **Dataset/Transform** (imagens, augmentation) e no **Model** (CNN em vez de MLP). O resto é o mesmo.

Dataset de imagens em pastas

Quando seu dataset é **uma pasta por classe**, `ImageFolder` faz quase tudo:

```
from torchvision import datasets, transforms

transform_treino = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                          std=[0.229, 0.224, 0.225]),
])

dataset = datasets.ImageFolder(root='dados/treino', transform=transform_treino)

dataset.classes # ['cao', 'gato', ...]
dataset.class_to_idx # {'cao': 0, 'gato': 1, ...}
img, lbl = dataset[0] # (tensor 3x224x224, int)
```

Estrutura esperada de pastas

`dados/treino/cao/img1.jpg`, `dados/treino/cao/img2.jpg`, `dados/treino/gato/img1.jpg`, etc. O `ImageFolder` usa o nome da subpasta como classe.

Ordem dos transforms

A ordem importa. Regra: **augmentations geométricas** → **ToTensor** → **Normalize**.

```
transforms.Compose([
    # 1. operacoes em PIL Image (entrada)
    transforms.Resize(256),
    transforms.CenterCrop(224), # ou RandomCrop no treino
    transforms.RandomHorizontalFlip(), # so faz sentido em algumas tasks
    transforms.ColorJitter(brightness=0.2, contrast=0.2),

    # 2. converter para Tensor (float em [0,1])
    transforms.ToTensor(),

    # 3. operacoes em Tensor
    transforms.Normalize(mean=MEAN, std=STD),
    transforms.RandomErasing(p=0.2), # so funciona em Tensor
])
```

Armadilha clássica

Normalize antes de **ToTensor** dá erro: ele espera um Tensor, não uma PIL. **RandomErasing** antes de **ToTensor** também.

Por que *data augmentation*?

Modelos profundos têm milhões de parâmetros e tendem a *decorar* o treino. Augmentation **cria variações sintéticas** a cada época, o modelo nunca vê exatamente a mesma imagem duas vezes.

✓ Reduz overfitting

✓ Melhora generalização

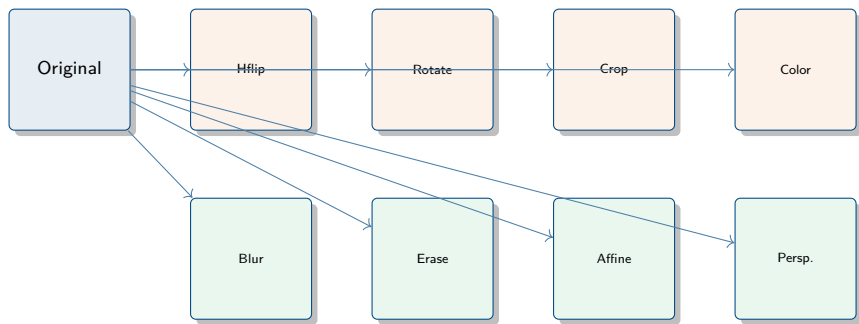
✓ Robustez a variações

✓ Sem custo de coletar mais dados

Regra de ouro

Augmentation é **só no treino**. Nunca no `val_loader` nem no `test_loader`. Validar com imagens distorcidas não mede o que você quer medir.

Augmentations comuns – visualização



Cada batch sorteia a um conjunto diferente dessas transformações. A rede aprende que “gato espelhado”, “gato rotacionado” e “gato cortado” ainda são gato. **Invariância aprendida pelos dados, não pela arquitetura.**

Pipeline de augmentation típico (CIFAR-10)

```
mean = [0.4914, 0.4822, 0.4465]
std = [0.2470, 0.2435, 0.2616]

transform_treino = transforms.Compose([
    transforms.RandomCrop(32, padding=4), # corta janela 32x32 dentro de 40x40
    transforms.RandomHorizontalFlip(),
    transforms.ColorJitter(0.2, 0.2, 0.2),
    transforms.ToTensor(),
    transforms.Normalize(mean, std),
    transforms.RandomErasing(p=0.1),
])

transform_val = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(mean, std), # SEM augmentations
])
```

Por que duas pipelines

Você cria **dois transforms diferentes** e passa cada um para o seu Dataset respectivo. `ImageFolder(treino, transform=t_treino)`, `ImageFolder(val, transform=t_val)`.

transforms.v2 – a versão moderna

Desde PyTorch 2.0+, há uma API mais rica em `torchvision.transforms.v2`. Diferenças principais: mais rápida, aceita batches, e funciona com máscaras de segmentação e bounding boxes.

```
from torchvision.transforms import v2

transform_treino = v2.Compose([
    v2.ToImage(), # PIL -> tv_tensors.Image
    v2.RandomResizedCrop(size=(224, 224), antialias=True),
    v2.RandomHorizontalFlip(p=0.5),
    v2.ToDtype(torch.float32, scale=True), # uint8 -> float32 / 255
    v2.Normalize(mean=[0.485, 0.456, 0.406],
                  std =[0.229, 0.224, 0.225]),
])
```

Quando migrar

Se você está começando um projeto novo hoje, use `v2`. É mais rápida e o mais antigo `transforms` entrará em *legacy* eventualmente.

CrossEntropyLoss – detalhes que importam

Você já usou. Vale revisar três pontos:

```
# 1. ESPERA LOGITS, nao probabilidades. NAO COLOQUE softmax na ultima camada!
out = modelo(x) # shape (N, num_classes) -- logits
loss = F.cross_entropy(out, y) # y: (N,) inteiros, NAO one-hot

# 2. Internamente faz LogSoftmax + NLLLoss em um passo (numericamente estavel)

# 3. weight para classes desbalanceadas
class_weights = torch.tensor([1.0, 2.5, 0.5, ...]).to(device)
criterio = nn.CrossEntropyLoss(weight=class_weights, label_smoothing=0.1)
```

Erros frequentes

- Colocar `nn.Softmax(dim=1)` no fim do modelo $\Rightarrow$ aplica softmax *duas* vezes, loss não desce.
- Passar one-hot em `y` $\Rightarrow$ erro ou resultado errado.
- Esquecer de mover `class_weights` para a GPU.

Otimizadores – fórmulas *de cabeça*

Todo otimizador atualiza θ a partir do gradiente $g_t = \nabla_{\theta} L(\theta_t)$. Diferença está no *como*.

Otimizador	Atualização	Quando
SGD	$\theta \leftarrow \theta - \eta g$	baseline simples
SGD + mom.	$v \leftarrow \mu v + g; \theta \leftarrow \theta - \eta v$	treina mais rápido
Adam	η por parâmetro via m_t, v_t	default seguro
AdamW	Adam + weight decay correto	ResNet, ViT

Recomendação pragmática

Para CNN nova, comece com **Adam(lr=1e-3)** ou **AdamW(lr=1e-3, weight_decay=1e-4)**. Para reprodução fiel de papers de ResNet/VGG, use **SGD(lr=0.1, momentum=0.9, weight_decay=5e-4)** + cosine schedule.

SGD vs Adam vs AdamW em código

```
# SGD classico com momentum + nesterov
opt = torch.optim.SGD(model.parameters(),
                       lr=0.1, momentum=0.9, nesterov=True,
                       weight_decay=5e-4)

# Adam (lr default 1e-3)
opt = torch.optim.Adam(model.parameters(),
                       lr=1e-3, betas=(0.9, 0.999), weight_decay=0)

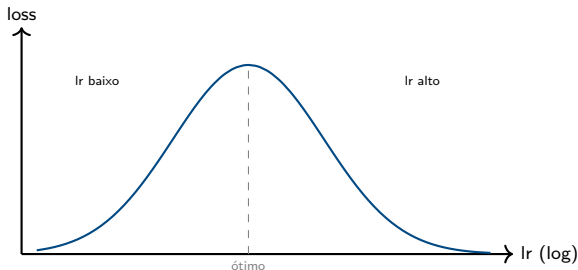
# AdamW (recomendado para arquiteturas modernas)
opt = torch.optim.AdamW(model.parameters(),
                        lr=1e-3, weight_decay=1e-2)
```

Diferença Adam x AdamW

Em Adam, weight decay vira parte do gradiente – e não se comporta como deveria. AdamW aplica weight decay **após** o update, como na fórmula original. *Para CNN moderna, prefira AdamW.*

Encontrando o *learning rate* ideal

Hiperparâmetro mais sensível. Pequeno demais = treino não avança. Grande demais = NaN.



Técnica do *lr range test* (`lr_finder`)

Treina por 100 iter. aumentando *exponencialmente* o lr de 10^{-7} até 10^1 . Plote loss vs lr (log). Escolha um lr **1 década antes do mínimo**. Pacote: `torch-lr-finder`.

Weight decay $\neq$ L2 (em todos os otimizadores)

L2 puro: você soma $\lambda \sum w_i^2$ na loss. O gradiente $-2\lambda w_i$ entra junto com os demais – inclusive passando pelo *momento* no Adam.

Weight decay desacoplado (AdamW): aplica $w_i \leftarrow w_i - \eta \lambda w_i$ *depois* do passo do otimizador, sem passar pelo momento.

```
# SGD: weight_decay equivale a L2 verdadeira
opt = torch.optim.SGD(modelo.parameters(), lr=0.1, weight_decay=1e-4)

# Adam: weight_decay e' L2 misturada com momento (NAD equivale)
opt = torch.optim.Adam(modelo.parameters(), lr=1e-3, weight_decay=1e-4)

# AdamW: weight_decay DESACOPLADO (jeito moderno; default na maioria
# das aplicacoes "modernas" de transfer learning)
opt = torch.optim.AdamW(modelo.parameters(), lr=1e-3, weight_decay=1e-2)
```

Detalhe importante

Por padrão, `weight_decay` regulariza *também* biases e parâmetros do BatchNorm (γ, β). Em muitos papers modernos **exclui-se** esses do decay (`param_groups`), porque encolher bias e γ piora o treino.

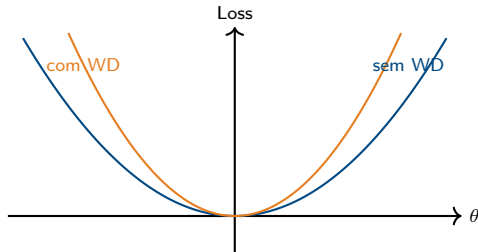
Fonte: Stevens et al., *DLPT*, p. 220 (definindo L2 = weight decay no SGD).

Weight decay – regularização L2

Acrescenta penalidade $\lambda \|\theta\|^2$ à loss. Equivalente, em SGD, a:

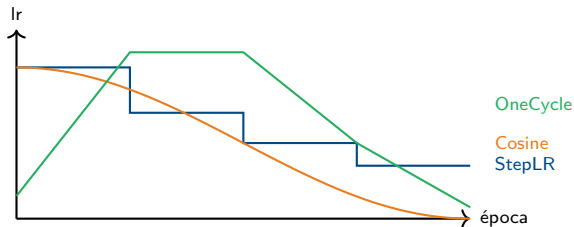
$$\theta \leftarrow \theta - \eta(g + 2\lambda\theta) = (1 - 2\eta\lambda)\theta - \eta g.$$

Cada passo encolhe θ um pouquinho em direção a zero.



Geometricamente, weight decay **aprofunda o poço** em $\theta = 0$: soluções com pesos menores ficam favorecidas. Generaliza melhor.

Schedulers de learning rate



StepLR: $\eta \times 0.1$ a cada K épocas. **CosineAnnealing:** cai suavemente como cosseno. **OneCycle:** sobe até um pico e desce – bem agressivo, mas funciona muito bem para treinos curtos.

Schedulers em código

```
from torch.optim.lr_scheduler import (
    StepLR, CosineAnnealingLR, OneCycleLR, ReduceLROnPlateau
)

# Decai 10x a cada 30 epocas
scheduler = StepLR(opt, step_size=30, gamma=0.1)

# Cosseno suave ao longo de T_max epocas
scheduler = CosineAnnealingLR(opt, T_max=epocas)

# OneCycle -- chama scheduler.step() POR BATCH, nao por epoca
scheduler = OneCycleLR(opt, max_lr=0.1,
                       steps_per_epoch=len(train_loader), epochs=epocas)

# Reduz quando metrica para de melhorar
scheduler = ReduceLROnPlateau(opt, mode='min', factor=0.5, patience=3)
# usa: scheduler.step(val_loss)
```

Sutileza

`OneCycleLR` e schedulers “por batch” chamam `step()` dentro do loop interno. `StepLR`, `CosineAnnealingLR` e `ReduceLROnPlateau` chamam após cada época.

Loop técnico completo – parte 1 (treino)

```
def treinar_epoca(model, loader, criterio, optimizer, device):
    model.train()
    soma_loss, acertos, total = 0.0, 0, 0
    for x, y in loader:
        x, y = x.to(device, non_blocking=True), y.to(device, non_blocking=True)
        optimizer.zero_grad()
        out = model(x)
        loss = criterio(out, y)
        loss.backward()
        optimizer.step()

        soma_loss += loss.item() * x.size(0)
        acertos += (out.argmax(1) == y).sum().item()
        total += x.size(0)
    return soma_loss / total, acertos / total
```

Boas práticas: `model.train()` no início, `non_blocking=True` se `pin_memory=True`, multiplicar `loss.item()` por `x.size(0)` para média correta (último batch pode ser menor).

Loop técnico completo – parte 2 (validação)

```
@torch.no_grad()
def avaliar(model, loader, criterio, device):
    model.eval()
    soma_loss, acertos, total = 0.0, 0, 0
    for x, y in loader:
        x, y = x.to(device, non_blocking=True), y.to(device, non_blocking=True)
        out = model(x)
        loss = criterio(out, y)
        soma_loss += loss.item() * x.size(0)
        acertos += (out.argmax(1) == y).sum().item()
        total += x.size(0)
    return soma_loss / total, acertos / total
```

Detalhes que pegam

`@torch.no_grad()` (ou `with torch.no_grad():`) **desliga autograd** – mais rápido e menos memória. `model.eval()` muda comportamento de BN e Dropout. Não chame `loss.backward()` nem `optimizer.step()` aqui.

Avaliar treino e validação – o padrão do livro

Truque pedagógico: rode a métrica nos *dois* loaders. A diferença `acc_train - acc_val` é seu indicador direto de *generalization gap*.

```
@torch.no_grad()
def validar(model, train_loader, val_loader, device):
    model.eval()
    resultados = {}
    for nome, loader in [("train", train_loader), ("val", val_loader)]:
        acertos, total = 0, 0
        for x, y in loader:
            x, y = x.to(device), y.to(device)
            pred = model(x).argmax(dim=1)
            acertos += (pred == y).sum().item()
            total += y.size(0)
        resultados[nome] = acertos / total
    print(f"Accuracy {nome}: {resultados[nome]:.4f}")
    return resultados
```

Como ler

`acc_train = 0.99, acc_val = 0.79` $\Rightarrow$ gap de 20 pts: **overfit clássico**. `acc_train = 0.85, acc_val = 0.83` $\Rightarrow$ underfit ou bem regularizado. Use esse número como cronómetro para parar de aumentar a capacidade.

Adaptado de Stevens et al., *DLPT*, p. 214.

Loop técnico completo – parte 3 (orquestração)

```
melhor_acc = 0.0
for epoca in range(epocas):
    tr_loss, tr_acc = treinar_epoca(model, train_loader, criterio, opt, device)
    vl_loss, vl_acc = avaliar(model, val_loader, criterio, device)
    scheduler.step() # ou scheduler.step(vl_loss)

    writer.add_scalars('Loss', {'treino': tr_loss, 'val': vl_loss}, epoca)
    writer.add_scalars('Acc', {'treino': tr_acc, 'val': vl_acc}, epoca)
    writer.add_scalar('lr', opt.param_groups[0]['lr'], epoca)

    if vl_acc > melhor_acc:
        melhor_acc = vl_acc
        torch.save({'epoca': epoca, 'model': model.state_dict(),
                    'opt': opt.state_dict(), 'best_acc': melhor_acc,
                    'melhor.pth'})
    print(f"E{epoca:03d} tr {tr_loss:.4f}/{tr_acc:.3f} "
          f"val {vl_loss:.4f}/{vl_acc:.3f}")
```

Barra de progresso com tqdm

Treinar 100 épocas sem feedback visual é ruim. `tqdm` é o padrão para barras de progresso:

```
from tqdm import tqdm

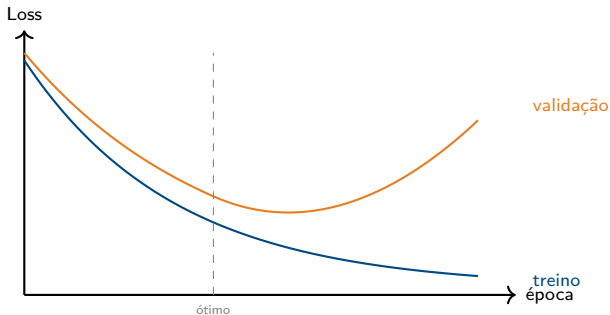
for epoca in range(EPOCAS):
    pbar = tqdm(train_loader, desc=f"Epoca {epoca:3d}")
    for x, y in pbar:
        x, y = x.to(device), y.to(device)
        opt.zero_grad()
        loss = criterio(modelo(x), y)
        loss.backward(); opt.step()

    # Atualizar barra com loss atual
    pbar.set_postfix(loss=f"{loss.item():.4f}",
                    lr =f"{opt.param_groups[0]['lr']:.2e}")
```

Em notebook (Colab/Jupyter)

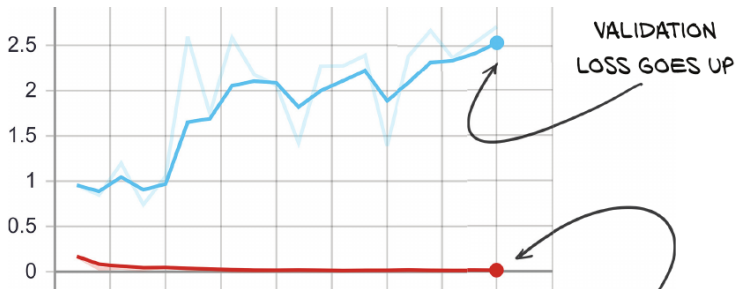
Use `from tqdm.notebook import tqdm` (mais bonita) ou `from tqdm.auto import tqdm` (escolhe sozinho terminal vs notebook).

Overfitting visual: curvas que se separam



Treino continua descendo, val sobe $\Rightarrow$ **overfitting**. Sinal de que você deve **parar aí** (*early stopping*) ou aplicar mais regularização.

Overfitting – exemplo real do livro



Training loss (vermelho) cai pra zero; validation loss (azul) **sobe**. O modelo decorou o treino, perdeu a capacidade de generalizar.

Soluções nas próximas pages: augmentation, dropout, weight decay, modelo menor, mais dados.

Fonte: Stevens, Antiga, Viehmann – *Deep Learning with PyTorch*, fig. 12.18, p. 344.

Como combater overfitting (checklist)

✓ Mais data augmentation

✓ Aumentar `weight_decay`

✓ Adicionar dropout

✓ Reduzir capacidade do modelo

✓ Early stopping

✓ Coletar mais dados

Ordem de tentativa prática: comece com mais augmentation; depois ajuste weight decay; depois dropout; depois mais dados; *por último* reduza o modelo. Reduzir o modelo é o último recurso porque você sacrifica capacidade.

A “receita” do livro: primeiro overfite, depois combata

Etapas (apud Stevens et al., p. 434 / Karpathy)

Etapas 1: aumente o modelo até *ele overfitar* (train loss $\rightarrow 0$, val loss subindo). *Só assim* você sabe que tem capacidade suficiente.

Etapas 2: *ai sim* comece a combater overfit – augmentation, dropout, weight decay, mais dados. Cada técnica empurra val um pouco para baixo.

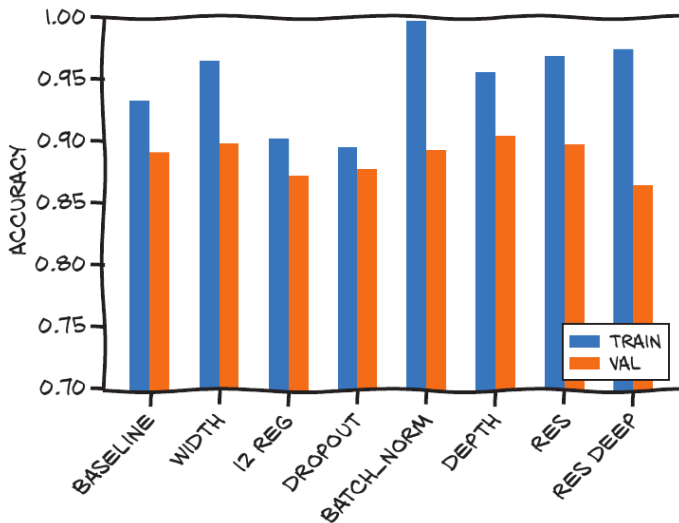
Não ordene o contrário: começar com modelo pequeno e muito regularizado camufla o problema – você fica sem saber se está em *underfit* ou em *overfit*.

Augmentação tem que ser não-trivial

Uma transformação que o modelo aprende a *ignorar* (ex.: mexer só em 4 pixels do canto) gera dataset 4 bilhões de vezes maior *sem nenhum ganho*. Boa augmentation preserva a classe mas **muda o que o modelo iria memorizar**.

(Stevens et al., *DLPT*, p. 347.)

Cada técnica em números



Mesmo dataset (CIFAR-10 subset), variando uma técnica de cada vez. A barra laranja é o que importa (validation acc): **BatchNorm**, **depth** e **ResNet** dão os maiores saltos; **L2** e **dropout** ajudam menos isoladamente.

Early stopping em PyTorch (mão na massa)

```
class EarlyStopping:
    def __init__(self, paciencia=5, delta=1e-4):
        self.paciencia = paciencia
        self.delta = delta
        self.melhor = float('inf')
        self.contador = 0
        self.parar = False

    def __call__(self, val_loss):
        if val_loss < self.melhor - self.delta:
            self.melhor = val_loss
            self.contador = 0
        else:
            self.contador += 1
            if self.contador >= self.paciencia:
                self.parar = True

# uso no loop:
es = EarlyStopping(paciencia=8)
for epoca in range(epocas):
    ...
    es(vl_loss)
    if es.parar:
        print(f"Parando na epoca {epoca}"); break
```

Checkpoint COMPLETO

Salvar apenas `state_dict` é o mínimo. Para retomar o treino *exatamente* de onde parou, salve mais:

```
def salvar_checkpoint(epoca, model, opt, scheduler, melhor_acc, caminho):
    torch.save({
        'epoca': epoca,
        'model_state': model.state_dict(),
        'optimizer_state': opt.state_dict(),
        'scheduler_state': scheduler.state_dict(),
        'melhor_acc': melhor_acc,
        'rng_state': torch.get_rng_state(), # opcional
    }, caminho)

def carregar_checkpoint(caminho, model, opt, scheduler):
    ck = torch.load(caminho, map_location='cpu')
    model.load_state_dict(ck['model_state'])
    opt.load_state_dict(ck['optimizer_state'])
    scheduler.load_state_dict(ck['scheduler_state'])
    return ck['epoca'], ck['melhor_acc']
```

Salvar melhor modelo + state_dict mínimo

Em deploy, geralmente *não* precisamos do otimizador. Salve dois arquivos:

```
# Durante o treino: checkpoint completo para retomada
salvar_checkpoint(epoca, model, opt, scheduler, melhor_acc, 'ck_full.pth')

# Para deploy: so o state_dict
torch.save(model.state_dict(), 'modelo_final.pth')

# Carregando em producao:
model = CIFAR10CNN(num_classes=10)
model.load_state_dict(torch.load('modelo_final.pth', map_location='cpu'))
model.eval()
```

Bom hábito

Sempre passe `map_location` em `torch.load` – garanta que carrega na CPU mesmo que tenha sido salvo na GPU. Você move para GPU depois com `.to(device)`.

TensorBoard avançado – add_scalar(s)

```
from torch.utils.tensorboard import SummaryWriter
writer = SummaryWriter(log_dir=f'runs/exp_{datetime.now():%Y%m%d_%H%M}')
```

Escalar simples

```
writer.add_scalar('treino/loss', tr_loss, epoca)
```

Dois escalares no mesmo plot

```
writer.add_scalars('loss', {'treino': tr_loss, 'val': vl_loss}, epoca)
```

Hiperparametros

```
writer.add_scalar('lr', opt.param_groups[0]['lr'], epoca)
writer.add_scalar('grad_norm', total_grad_norm, epoca)
```

writer.flush() # garante escrita em disco

writer.close() # no fim

Convenção de nomes

Use *categoria/nome* (com barra). O TensorBoard agrupa automaticamente: *loss/treino* e *loss/val* aparecem juntos.

TensorBoard – imagens, histogramas, grafo

```
# Grid de imagens (batch atual)
from torchvision.utils import make_grid
grid = make_grid(imgs[:16], nrow=4, normalize=True)
writer.add_image('batch/treino', grid, epoca)

# Distribuicao dos pesos -- excelente para detectar treino estagnado
for nome, p in model.named_parameters():
    writer.add_histogram(nome, p, epoca)
    if p.grad is not None:
        writer.add_histogram(f'{nome}.grad', p.grad, epoca)

# Grafo do modelo (uma vez so)
writer.add_graph(model, imgs[:1])

# Curva precision-recall por classe
writer.add_pr_curve('pr/classe_0', y_true_bin, y_score, epoca)
```

Visão geral

Ao final de um treino, você olha 4 abas no TensorBoard: **Scalars** (loss, acc, lr), **Images** (sanity check do augmentation), **Histograms** (gradientes vivos?), **Graph** (estrutura do modelo).

Matriz de confusão e métricas por classe

```
from sklearn.metrics import confusion_matrix, classification_report
import numpy as np

@torch.no_grad()
def coletar_predicoes(model, loader, device):
    y_true, y_pred = [], []
    model.eval()
    for x, y in loader:
        out = model(x.to(device))
        y_true.extend(y.numpy())
        y_pred.extend(out.argmax(1).cpu().numpy())
    return np.array(y_true), np.array(y_pred)

y_true, y_pred = coletar_predicoes(model, val_loader, device)
print(confusion_matrix(y_true, y_pred))
print(classification_report(y_true, y_pred, target_names=dataset.classes))
```

Olhar acurácia agregada esconde problemas. *Quais classes o modelo erra mais? Onde está confundindo?* Em CIFAR-10 é clássico confundir gato/cachorro e carro/caminhão.

Classes desbalanceadas no treino

Se “cachorro” tem 10000 imagens e “hamster” tem 100, o modelo aprende a sempre dizer “cachorro”. Duas opções:

```
# Op A: peso na loss
class_weights = torch.tensor([1/n for n in contagens_por_classe]).to(device)
class_weights = class_weights / class_weights.sum() * num_classes
criterio = nn.CrossEntropyLoss(weight=class_weights)

# Op B: WeightedRandomSampler (cada classe igualmente vista)
from torch.utils.data import WeightedRandomSampler
peso_amostra = [1.0/contagens_por_classe[lbl] for lbl in dataset.targets]
sampler = WeightedRandomSampler(peso_amostra, num_samples=len(dataset),
                                replacement=True)
loader = DataLoader(dataset, batch_size=64, sampler=sampler) # nao usar shuffle!
```

Qual escolher?

`class_weight` é mais simples e funciona em qualquer caso. `WeightedRandomSampler` é necessário quando uma classe é *muito* rara (1% ou menos).

DataLoader otimizado

```
train_loader = DataLoader(  
    dataset_treino,  
    batch_size=128,  
    shuffle=True,  
    num_workers=4, # processos paralelos para carregar  
    pin_memory=True, # acelera transferencia CPU->GPU  
    persistent_workers=True, # nao recriar workers a cada epoca  
    drop_last=True, # descarta ultimo batch incompleto  
)
```

Regras práticas

- `num_workers = ~#CPUs/2` (4-8 é comum). Mais é desperdício.
- `pin_memory=True` só ajuda se você usa GPU.
- `persistent_workers=True` acelera épocas curtas (CIFAR-10).
- `drop_last=True` no treino para que BatchNorm não receba batch minúsculo no fim.

Reprodutibilidade – seeds e determinismo

```
import random, numpy as np, torch, os

def fixar_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    os.environ['PYTHONHASHSEED'] = str(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False # mais lento, mais reprodutivel

fixar_seed(42)

# Para DataLoader deterministico:
g = torch.Generator(); g.manual_seed(42)
loader = DataLoader(ds, batch_size=64, shuffle=True, generator=g)
```

Preco do determinismo

Algumas operações em CUDA não são determinísticas. `torch.use_deterministic_algorithms(True)` força – mas pode dar erro em operações sem implementação determinística. Em produção, basta `seeds` e `cudnn.deterministic=True`.

Gradient clipping (estabilizar treino)

Quando o gradiente *explode*, a loss vira NaN. Solução: recortar a norma:

```
# Após loss.backward(), ANTES de optimizer.step()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

optimizer.step()
```

Quando ativar

Comum em RNNs e Transformers. Em CNNs é raro precisar – mas se a loss virar NaN no início do treino, ativar `clip_grad_norm` com `max_norm=1.0` costuma resolver.

Diferença do `clip_grad_value_`

`clip_grad_norm` normaliza o vetor de gradientes inteiro. `clip_grad_value_` recorta componente a componente. Norma é geralmente preferida.

Quando a loss vira NaN – checklist

LR muito alto (tente $lr/10$)

Esqueceu de normalizar a entrada

Divisão por zero / log de zero na loss custom

Ativação explodindo (clipping resolve)

Como diagnosticar: imprima `loss.item()` a cada K batches. Imprima a norma do gradiente: `total = sum(p.grad.norm() for p in model.parameters() if p.grad is not None)`. Se norma cresce sem parar antes do NaN, você tem explosão → clipping.

Resumo do Bloco 3

Augmentation
só no treino

TensorBoard
scalars, images, hist

Early stopping
parar quando val piora

Schedulers
Step, Cosine, OneCycle

Checkpoint
model+opt+sched

Reprodutibilidade
seeds e cudnn

Próximo bloco: transfer learning sistemático, visualização de filtros, Grad-CAM, mixed precision.



Tópicos avançados e prática

Transfer learning – a motivação

Treinar do zero uma ResNet-50 em ImageNet leva vários dias em uma GPU boa. Mas as features que ela aprende (bordas, texturas, partes) são **genéricas** – servem para quase qualquer tarefa visual.



Ideia: a primeira metade da rede pré-treinada já “sabe ver”. Você só ajusta o final para a sua tarefa.
Resultado: treinos rápidos, com poucos dados, e melhores que do zero.

Dois modos de transfer learning

Feature extraction

backbone **congelado**
só a cabeça treina
rápido, pouco dado

Fine-tuning

backbone **atualizado**
lr menor que no normal
lento, mais dado

Diferença técnica: em feature extraction, `param.requires_grad = False` no backbone. Em fine-tuning, todos os parâmetros recebem gradiente – mas você usa *learning rate menor* (tipicamente 10× menor) para não destruir o que já foi aprendido.

Quando usar cada modo

Dois eixos: *tamanho do seu dataset* e *similaridade com ImageNet*.

Decisão rápida

Dataset pequeno
+ similar a ImageNet
Feature extraction

Dataset pequeno
+ diferente
Feature extr. ou fine-tune
de camadas finais

Dataset grande
+ similar
Fine-tuning completo
com lr baixo

Dataset grande
+ diferente
Fine-tuning + lr maior
(talvez do zero)

Regra de bolso: *quanto mais dado e mais diferente do ImageNet, mais você atualiza*. Quase nunca treinar do zero é a melhor opção quando você tem ImageNet pré-treinado disponível.

Carregando modelo pré-treinado (PyTorch moderno)

```
from torchvision import models
from torchvision.models import ResNet18_Weights, ResNet50_Weights, VGG16_Weights

# Sintaxe NOVA (PyTorch >= 0.13) -- escolhe pesos explicitamente
modelo = models.resnet18(weights=ResNet18_Weights.IMAGENET1K_V1)
# ou: models.resnet18(weights="DEFAULT") # ultimo melhor disponivel
# ou: models.resnet18(weights=None) # sem pre-treino (do zero)

# Sintaxe ANTIGA (deprecada, ainda funciona)
modelo = models.resnet18(pretrained=True) # gera DeprecationWarning

# Pegando as transforms certas que combinam com o pre-treino
preprocess = ResNet18_Weights.IMAGENET1K_V1.transforms()
```

Use as transforms “oficiais”

Cada modelo pré-treinado tem `weights.transforms()` que retorna o pipeline correto (resize, normalize, etc). Use isso em vez de copiar valores – se o modelo foi treinado com normalização diferente, você saca sozinho.

Feature extraction – congelando o backbone

```
modelo = models.resnet18(weights="DEFAULT")

# 1. Congela tudo
for p in modelo.parameters():
    p.requires_grad = False

# 2. Substitui a cabeça (fc) por uma nova, adaptada a sua tarefa
num_ftrs = modelo.fc.in_features # 512 para resnet18
modelo.fc = nn.Linear(num_ftrs, num_classes_minhas)

# 3. So a nova fc tem requires_grad = True (padrao em camada nova)
modelo = modelo.to(device)

# 4. Otimiza SO os parametros da fc
opt = torch.optim.Adam(modelo.fc.parameters(), lr=1e-3)
```

Por que treinar tão rápido

Você treina umas centenas de pesos (a `fc` nova), não os 11M da ResNet. Cada época é $\sim 10\times$ mais rápida que o fine-tuning completo.

Fine-tuning – atualizando tudo com lr menor

```
modelo = models.resnet18(weights="DEFAULT")

# 1. NAO congela -- todos os parametros sao treinaveis
# 2. So substitui a fc
num_fts = modelo.fc.in_features
modelo.fc = nn.Linear(num_fts, num_classes_minhas)
modelo = modelo.to(device)

# 3. lr menor para nao destruir o que ja foi aprendido
opt = torch.optim.Adam(modelo.parameters(), lr=1e-4) # 10x menor

# 4. Opcao avancada: lr diferente por grupo de parametros
opt = torch.optim.Adam([
    {'params': modelo.layer1.parameters(), 'lr': 1e-5}, # camada 1 lenta
    {'params': modelo.layer2.parameters(), 'lr': 5e-5},
    {'params': modelo.layer3.parameters(), 'lr': 1e-4},
    {'params': modelo.layer4.parameters(), 'lr': 5e-4}, # camada 4 mais agil
    {'params': modelo.fc.parameters(), 'lr': 1e-3}, # cabeca rapida
])
```

Onde é a “cabeça” em cada modelo

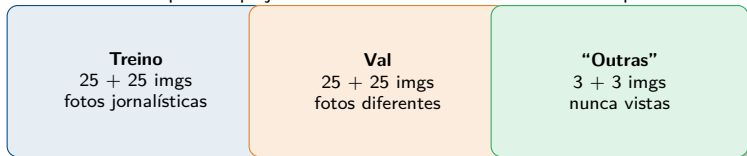
O nome da última camada muda de modelo para modelo. Para *transfer learning*, precisa saber onde mexer:

Modelo	Cabeça final	Features de entrada
resnet18/50	model.fc	512 / 2048
vgg16	model.classifier[6]	4096
alexnet	model.classifier[6]	4096
densenet121	model.classifier	1024
efficientnet_b0	model.classifier[1]	1280
mobilenet_v3	model.classifier[3]	1024

```
# Inspeção antes
print(modelo) # mostra a estrutura toda
print(modelo.classifier) # ve apenas a head do VGG
```

Aplicado: classificar Bolsonaro × Lula

Vamos resolver um problema **real e pequeno**: dadas fotos das duas pessoas, treinar um classificador binário. É o cenário típico em projetos reais – você tem dados rotulados aos poucos.



Desafio

50 imagens de treino é **minimúsculo** para treinar uma CNN do zero – o modelo decora. Transfer learning é *obrigatório* aqui. Vamos comparar 3 estratégias e ver qual ganha.

O formato ImageFolder

Coloca cada classe numa subpasta. O `ImageFolder` lê automaticamente, atribuindo rótulos pela ordem alfabética dos diretórios.

```
data/bolsolula/  
  train/ train_cropped/ # faces, via MTCNN  
    bolsonaro/*.jpg bolsonaro/*.png  
    lula/*.jpg lula/*.png  
  val/ val_cropped/  
    bolsonaro/*.jpg bolsonaro/*.png  
    lula/*.jpg lula/*.png  
  outras/ outras_cropped/ # teste mundo real
```

```
from torchvision import datasets, transforms  
  
train_ds = datasets.ImageFolder(  
    "data/bolsolula/train",  
    transform=transform_train  
)  
print(train_ds.classes) # ['bolsonaro', 'lula'] -> idx 0 e 1
```

Pipeline de transforms (faces inteiras, 224x224)

```
IMNET_MEAN = (0.485, 0.456, 0.406) # estatísticas do ImageNet
IMNET_STD = (0.229, 0.224, 0.225) # (a ResNet espera isso)

transform_train = transforms.Compose([
    transforms.RandomResizedCrop((224, 224), scale=(0.7, 1.0)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(15),
    transforms.ToTensor(),
    transforms.Normalize(IMNET_MEAN, IMNET_STD),
])

transform_eval = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(IMNET_MEAN, IMNET_STD), # SEM augmentation
])
```

Detalhe importante

Usamos **mean/std do ImageNet** porque a ResNet foi treinada vendo dados normalizados assim. Mudar isso confunde o backbone pré-treinado.

Modo 1: ResNet-18 + feature extraction

```
from torchvision import models
from torchvision.models import ResNet18_Weights

def setup_resnet_fe(num_classes=2):
    model = models.resnet18(weights=ResNet18_Weights.DEFAULT)
    for p in model.parameters():
        p.requires_grad = False # congela TUDO
    in_feats = model.fc.in_features # 512
    model.fc = nn.Linear(in_feats, num_classes) # so a fc treina
    return model

modelo = setup_resnet_fe().to(device)
opt = optim.SGD(filter(lambda p: p.requires_grad, modelo.parameters()),
                 lr=1e-3, momentum=0.9)

# Parametros totais: ~11.2M
# Parametros treinaveis: ~1k (apenas 512*2 + 2)
```

Hipótese

As features “genéricas” da ResNet (bordas, texturas, formas) são boas o suficiente para separar Bolsonaro e Lula?

Modo 2: ResNet-18 + fine-tuning de layer4

```
def setup_resnet_ft(num_classes=2):
    model = models.resnet18(weights=ResNet18_Weights.DEFAULT)
    for p in model.parameters():
        p.requires_grad = False
    # Descongela layer4 (ultima stage convolucional) + fc
    for p in model.layer4.parameters():
        p.requires_grad = True
    in_feats = model.fc.in_features
    model.fc = nn.Linear(in_feats, num_classes)
    return model

modelo = setup_resnet_ft().to(device)
opt = optim.SGD(filter(lambda p: p.requires_grad, modelo.parameters()),
                 lr=1e-4, momentum=0.9) # lr 10x MENOR
# Parametros treinaveis: ~5M (toda a layer4 + a fc)
```

Por que descongelar só layer4?

As primeiras camadas (bordas, texturas) são quase universais. layer4 é onde a rede aprende “partes” específicas – bom ponto para adaptar a um novo domínio.

Por que ImageNet pode não bastar para faces

ImageNet tem cachorro, carro, garrafa, bicicleta... **nenhuma classe é "rosto humano"**. As features aprendidas servem para distinguir *categorias visuais*, não *peessoas*.

ImageNet
categorias genéricas
1M+ imgs

Problema
não sabe "rosto"
só "pessoa"

VGGFace2
3.3M fotos
9k identidades

Saída

Trocar o backbone por um treinado em **faces**: o **InceptionResnetV1** (**facenet-pytorch**) pré-treinado em **VGGFace2**. Suas features já separam *peessoas*.

Modo 3: InceptionResnetV1 + faces cortadas

Antes de mais nada, recortamos o rosto com o **MTCNN** ([facenet-pytorch](#)). Removemos fundo, roupa, contexto – só sobra o que importa: o rosto.

```
from facenet_pytorch import MTCNN, InceptionResnetV1

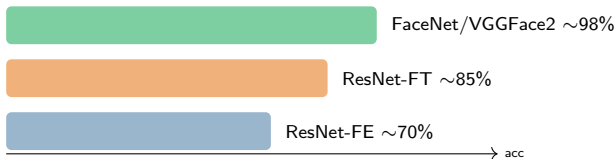
# Uma unica passada para gerar train_cropped/, val_cropped/...
mtcnn = MTCNN(image_size=160, margin=0, device=device)
# ... loop que salva os crops em data/bolsolula/*_cropped/

# Backbone treinado em VGGFace2; ja vem com a logits final
modelo = InceptionResnetV1(pretrained='vggface2', classify=True,
                           num_classes=2)
for p in modelo.parameters(): p.requires_grad = False
for p in modelo.logits.parameters(): p.requires_grad = True # so logits
modelo = modelo.to(device)

opt = optim.SGD(filter(lambda p: p.requires_grad, modelo.parameters()),
                 lr=1e-3, momentum=0.9)
```

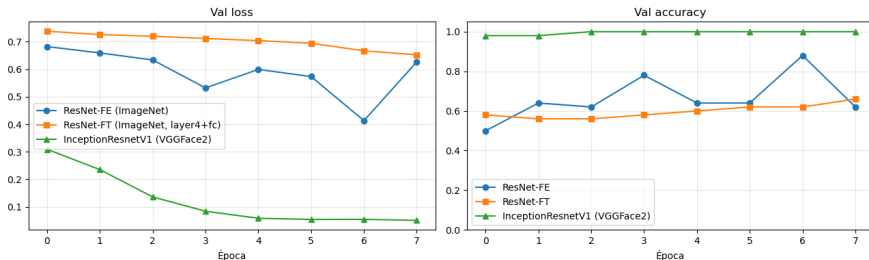
Resultado dos três modos lado a lado

Modo	Pré-treino	Param. trein.	Val Acc
1. ResNet-18 (FE)	ImageNet	~1k	~0.70
2. ResNet-18 (FT layer4+fc)	ImageNet	~5M	~0.85
3. InceptionResnetV1 (FE)	VGGFace2	~3k	~0.98



Os números variam por seed/épocas, mas a *ordem* é robusta. O modo 3 dispara na frente.

Curvas reais do experimento (notebook 03)



Verde (InceptionResnetV1/VGGFace2) cai pra *loss* ~ 0.05 e acerta $\sim 100\%$ já na 1^a época. **Azul/laranja** (ResNet/ImageNet) lutam para sair de 60-70% e ficam ruidosos.

O que isso confirma

O domínio do pré-treino importa *muito mais* do que o tamanho da rede ou o número de parâmetros treinados. (Saída real do `03_transfer_learning.ipynb`.)

Teste no “mundo real” – pasta outras/



6 imagens **que nunca entraram em treino ou validação**. Modelo FaceNet/VGGFace2 acerta com confiança alta. Val acc alta *não* prova generalização – avaliar em conjunto fora é a melhor evidência.

Saída real do [03_transfer_learning.ipynb](#).

Lição: o domínio do pré-treino importa

Três “transfer learnings” diferentes; o que mais ajudou não foi *quantos parâmetros* você treina, mas a **coerência entre o pré-treino e a sua tarefa**.

Heurística

- 1 Identifique o *tipo de coisa* que você vai classificar (objeto genérico, rosto, célula, satélite, texto na imagem. . .).
- 2 Procure um modelo pré-treinado *nesse* domínio (HuggingFace, timm, paperswithcode).
- 3 Só depois decida entre feature extraction ou fine-tuning.

Pré-processamento importa tanto quanto backbone

No modo 3, parte do ganho veio do **MTCNN**: tirar fundo e roupa concentra o sinal onde ele *está*. Boa prática: gastar tempo no pipeline antes de gastar em rede maior.

Visualizando os pesos da primeira camada conv

A primeira conv aprende detectores de baixo nível (bordas, texturas, cores). São $3 \times K \times K$ por kernel – dá para ver como imagem RGB.

```
import matplotlib.pyplot as plt
from torchvision.utils import make_grid

# Pega o tensor de pesos: (out_channels, in_channels, kH, kW)
filtros = modelo.conv1.weight.data.clone()
print(filtros.shape) # torch.Size([64, 3, 7, 7]) para resnet18

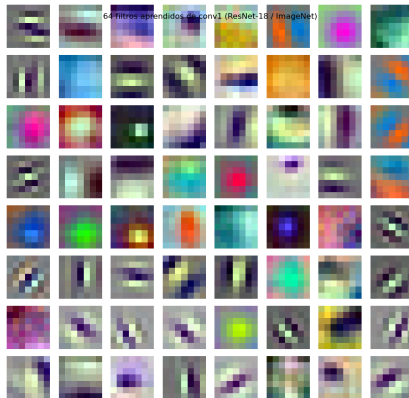
# Normalizar para [0,1] para visualizar
filtros = (filtros - filtros.min()) / (filtros.max() - filtros.min())

grid = make_grid(filtros, nrow=8, padding=1)
plt.figure(figsize=(10,10))
plt.imshow(grid.permute(1,2,0)) # CHW -> HWC
plt.axis('off'); plt.show()
```

O que você vê

Em ResNet-18 pré-treinada, conv1 mostra bordas em diversas orientações, manchas coloridas e gradientes – igualzinho ao córtex visual primário de mamíferos.

Os 64 filtros aprendidos da conv1 (ResNet-18)



Cada quadrado é um filtro $3 \times 7 \times 7$ – visível como imagem RGB porque tem 3 canais de entrada. Você vê **bordas direcionais**, **manchas**, **gradientes de cor**: detectores de baixo nível emergem *sozinhos* do treino em ImageNet.

Saída real do `03_transfer_learning.ipynb`.

Visualizando feature maps internos com hooks

```
ativacoes = {}

def hook_func(nome):
    def hook(module, entrada, saida):
        ativacoes[nome] = saida.detach()
    return hook

# Registra hook na camada que queremos espiar
modelo.layer1.register_forward_hook(hook_func('layer1'))
modelo.layer2.register_forward_hook(hook_func('layer2'))

# Forward normal
with torch.no_grad():
    _ = modelo(imagem.unsqueeze(0).to(device))

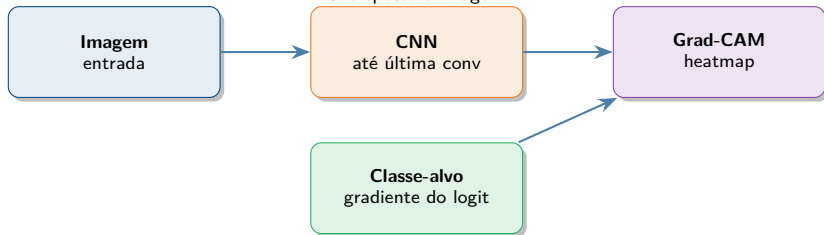
# Agora ativacoes['layer1'] tem o feature map da layer1
fmaps = ativacoes['layer1'][0] # (canais, H, W)
print(fmaps.shape) # ex.: torch.Size([64, 56, 56])
```

Diferença entre pesos e feature maps

Pesos (conv1.weight): o filtro em si. **Feature maps** (saídas das camadas): a resposta do filtro para uma imagem específica. São coisas diferentes.

Grad-CAM – “onde a rede está olhando?”

Quero saber quais pixels da entrada influenciaram a decisão do modelo. **Grad-CAM** dá um mapa de calor sobreposto à imagem.



Receita: tome a última conv. Pegue os gradientes da classe-alvo em relação a essa conv. Calcule a média espacial $\rightarrow$ pesos α_k . Faça $\sum_k \alpha_k A^k$ (soma ponderada dos canais). Passe ReLU. *Voilà*: mapa de importância.

Grad-CAM em código (versão didática)

```
import torch.nn.functional as F

ativacao, gradiente = {}, {}

def f_fwd(m, e, s): ativacao['x'] = s
def f_bwd(m, ge, gs): gradiente['x'] = gs[0]

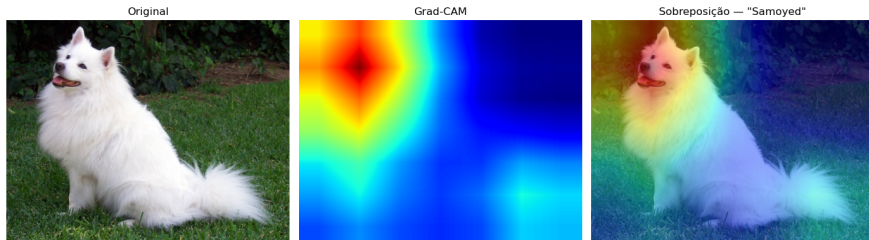
# Hook na última conv
modelo.layer4.register_forward_hook(f_fwd)
modelo.layer4.register_full_backward_hook(f_bwd)

# Forward + backward para a classe-alvo
out = modelo(x)
classe = out.argmax(1).item()
modelo.zero_grad()
out[0, classe].backward()

A = ativacao['x'][0] # (C, H, W)
g = gradiente['x'][0] # (C, H, W)
alpha = g.mean(dim=(1,2)) # (C,) pesos por canal
cam = F.relu((alpha[:,None,None] * A).sum(dim=0)) # (H, W)
cam = (cam - cam.min()) / (cam.max() - cam.min() + 1e-8)
```

Reescale a saída para o tamanho da imagem ([F.interpolate](#)) e sobreponha com transparência. Bibliotecas: [torchcam](#), [grad-cam](#) (pacote).

Grad-CAM em ação – saída real do notebook 04



Esquerda: imagem original. Meio: *heatmap* produzido pelo Grad-CAM mostrando **onde a rede olhou** para decidir a classe. Direita: sobreposição. O “calor” se concentra no **rosto e corpo do cão** – regiões que carregam a informação da classe.

Para que serve

Debug visual: se o Grad-CAM “acende” o fundo em vez do objeto, sua rede está aprendendo *shortcut features* – e vai falhar em uma imagem com fundo diferente. (Saída real do [04_visualizacao_e_debug.ipynb](#).)

Grad-CAM via biblioteca torchcam

Em produção, prefira a biblioteca pronta. Bem testada:

```
# pip install torchcam
from torchcam.methods import GradCAM
from torchcam.utils import overlay_mask
from torchvision.transforms.functional import to_pil_image

cam_extractor = GradCAM(modelo, target_layer='layer4')

# Forward + pegar mapa
out = modelo(input_tensor.unsqueeze(0))
classe = out.argmax(1).item()
mapas = cam_extractor(class_idx=classe, scores=out)
# mapas[0] tem shape (H', W')

# Sobrepor na imagem original
heatmap = to_pil_image(mapas[0].squeeze(), mode='F')
sobreposicao = overlay_mask(imagem_pil, heatmap, alpha=0.5)
sobreposicao.show()
```

Outras técnicas no torchcam

CAM, Grad-CAM, Grad-CAM++, Score-CAM, Smooth-Grad-CAM++. Para CNN clássica, Grad-CAM resolve. Para Transformers visuais (ViT), *Attention rollout* é mais apropriado.

GPU em PyTorch – boas práticas

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# Mover modelo e dados para GPU
modelo = modelo.to(device)
x = x.to(device, non_blocking=True) # nao bloqueia CPU enquanto copia
y = y.to(device, non_blocking=True)

# Verificar memoria
print(f"alocada: {torch.cuda.memory_allocated()/1e9:.2f} GB")
print(f"reservada: {torch.cuda.memory_reserved()/1e9:.2f} GB")

# Limpar cache (raramente necessario, mas util em debug)
torch.cuda.empty_cache()
```

Combo mágico

`pin_memory=True` no `DataLoader` + `non_blocking=True` em `.to(device)` faz a transferência CPU → GPU acontecer em paralelo com o próximo forward. Acelera 10-30% em datasets pequenos.

Mixed precision – treinar em float16

Por padrão, PyTorch usa `float32` (4 bytes por parâmetro). GPUs modernas (Volta, Turing, Ampere) têm hardware especial para `float16`, que é **2-3x mais rápido** e usa **metade da memória**.

`float32`

precisão alta
4 bytes/peso
estabilidade numérica

`float16`

precisão menor
2 bytes/peso
2-3x mais rápido

Mixed precision: *computações* em `float16`, mas *pesos e gradientes acumulados* em `float32`. Usar `torch.cuda.amp` faz isso automaticamente.

Mixed precision em código – mínimo

```
from torch.cuda.amp import autocast, GradScaler

scaler = GradScaler()

for x, y in loader:
    x, y = x.to(device, non_blocking=True), y.to(device, non_blocking=True)
    optimizer.zero_grad()

    with autocast(): # forward em float16
        out = model(x)
        loss = criterio(out, y)

    scaler.scale(loss).backward() # escala gradientes p/ float16
    scaler.step(optimizer) # update + descale
    scaler.update() # ajusta a escala
```

Por que o GradScaler

Gradientes em float16 podem virar zero (*underflow*). O `GradScaler` multiplica a loss por uma constante grande, calcula o gradiente, depois divide. Você não precisa pensar nisso – `amp` ajusta sozinho.

AMP integrado ao loop de treino

```
def treinar_epoca_amp(model, loader, criterio, opt, scaler, device):
    model.train()
    soma_loss, acertos, total = 0.0, 0, 0
    for x, y in loader:
        x = x.to(device, non_blocking=True)
        y = y.to(device, non_blocking=True)
        opt.zero_grad(set_to_none=True) # set_to_none > zerar

        with torch.cuda.amp.autocast():
            out = model(x)
            loss = criterio(out, y)

        scaler.scale(loss).backward()
        scaler.unscale_(opt) # opcional: para clipping
        torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        scaler.step(opt)
        scaler.update()

        soma_loss += loss.item() * x.size(0)
        acertos += (out.argmax(1) == y).sum().item()
        total += x.size(0)
    return soma_loss/total, acertos/total
```

torch.compile (PyTorch 2.x)

```
modelo = CIFAR10CNN()  
modelo = torch.compile(modelo) # 1 linha, pode dar 20-50% de speedup  
  
# Modos  
modelo = torch.compile(modelo, mode="default") # equilibrado  
modelo = torch.compile(modelo, mode="reduce-overhead") # bom p/ batches pequenos  
modelo = torch.compile(modelo, mode="max-autotune") # mais lento p/ compilar
```

Como funciona

O PyTorch 2.x transforma o seu `forward` em um grafo otimizado (via Triton/TorchInductor). Primeira execução demora mais (compilação); as seguintes ficam muito mais rápidas.

Quando não usar

Se você muda shapes dinamicamente a cada batch, ou se a sua rede tem muito controle de fluxo Python, `compile` recompila o tempo todo e *piora*. Em CNNs clássicas funciona muito bem.

Checklist técnico final

✓ `model.train()/eval()` nos lugares certos

✓ Augmentation só no treino

✓ Normalização igual no treino e na inferência

✓ Best checkpoint por validação

✓ Seeds + `cuda.deterministic`

✓ `pin_memory` + `num_workers` configurados

✓ TensorBoard ativo (scalars + histograms)

Esses 7 itens cobrem 95% das diferenças entre “código escolar” e “código de projeto real”. Vale conferir mentalmente em cada novo treino.

Erros comuns – e como detectar

Sintoma	Causa provável
Loss vira NaN Acurácia trava em $1/N$	lr alto, normalização esquecida, divisão por zero rede não está aprendendo nada – bug no shape ou no loss
Acurácia treino $\gg$ val Acurácia val maior que treino Treino lento	overfitting – mais aug., dropout, WD dropout muito agressivo OU bug no <code>eval()</code> <code>num_workers=0</code> , sem <code>pin_memory</code> , batch minúsculo
Out of memory Resultados mudam a cada run Output diferente de <code>eval()</code>	batch grande demais – reduza ou ative AMP faltam seeds, cudnn não-determinístico ainda há dropout/BN ativos – chame <code>model.eval()</code>

Truque mental

Antes de mexer no modelo, sempre rode `1 epoch` e olhe a loss: se ela não decresce, **problema está no pipeline**, não na rede.

Esqueleto de treino técnico (*tudo junto*)

```
fixar_seed(42)
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

train_loader = DataLoader(ds_tr, batch_size=128, shuffle=True,
                           num_workers=4, pin_memory=True, persistent_workers=True, drop_last=True)
val_loader = DataLoader(ds_vl, batch_size=256, num_workers=4, pin_memory=True)

modelo = CIFAR10CNN().to(device)
criterio = nn.CrossEntropyLoss(label_smoothing=0.05)
opt = torch.optim.AdamW(modelo.parameters(), lr=1e-3, weight_decay=1e-4)
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=EPOCHAS)
scaler = torch.cuda.amp.GradScaler()
writer = SummaryWriter('runs/cifar10_v1')
es = EarlyStopping(paciencia=10)

melhor = 0.0
for ep in range(EPOCHAS):
    tr_loss, tr_acc = treinar_epoca_amp(modelo, train_loader, criterio, opt, scaler, device)
    vl_loss, vl_acc = avaliar(modelo, val_loader, criterio, device)
    scheduler.step()
    writer.add_scalars('loss', {'tr': tr_loss, 'vl': vl_loss}, ep)
    if vl_acc > melhor: melhor = vl_acc; torch.save(modelo.state_dict(), 'best.pth')
    es(vl_loss)
    if es.parar: break
```

Quais hiperparâmetros realmente importam

Hiperpar.	Impacto	Faixa típica
learning rate	Crítico	10^{-4} a 10^{-1} (log)
batch size	Alto	32 a 512 (pot. de 2)
weight decay	Alto	10^{-5} a 10^{-2}
dropout (FC)	Médio	0.1 a 0.5
n. de épocas	Médio	30 a 200 (CIFAR), 90+ (ImageNet)
augmentations	Médio	2-5 transformacoes
arquitetura	Variavel	influência menor que LR/aug

Conselho prático

Comece sempre tunando o lr. 80% das melhorias vêm daí. Use ferramentas como [optuna](#) ou [wandb sweeps](#) para busca sistemática.

Indo para produção – TorchScript / ONNX

Para deploy fora do PyTorch, exporta a rede num formato neutro:

```
# TorchScript (deploy em C++ via libtorch)
modelo.eval()
exemplo = torch.randn(1, 3, 224, 224)
scripted = torch.jit.trace(modelo, exemplo)
scripted.save('modelo.pt')

# ONNX (deploy em qualquer runtime: TensorRT, OpenVINO, browser)
torch.onnx.export(modelo, exemplo, 'modelo.onnx',
    input_names=['img'], output_names=['logits'],
    dynamic_axes={'img': {0: 'batch'}, 'logits': {0: 'batch'}},
    opset_version=17)
```

Quantização para celular/embarcado

`torch.ao.quantization` permite reduzir pesos para `int8` – modelo 4x menor, 2-4x mais rápido em CPU. É padrão em modelos para Android/iOS.

Ponte para a próxima aula – detecção

Classificação responde “o que tem na imagem?”. A próxima fronteira: “*o que tem e onde está?*”.

Classificação
“cachorro”

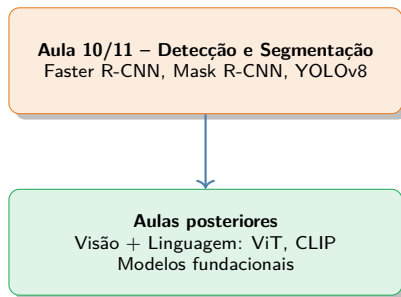
Detecção
“cachorro em (x,y,w,h) ”

Segmentação
máscara pixel-a-pixel

Pose / Keypoints
esqueleto da figura

Todas usam CNNs (ou Transformers visuais) como *backbone*, seguidas de cabeças especializadas. As redes que vocês viram hoje são a **base** de Faster R-CNN, Mask R-CNN, YOLO e DETR.

O que vem depois



As duas têm em comum o que você aprendeu hoje: convolução, pooling, BatchNorm, **transfer learning**. O esqueleto continua o mesmo; muda a cabeça (boxes, máscaras, embeddings) e o tipo de loss.

Recapitulando o dia inteiro

1

Fundamentos da convolução
 K , P , S , D , BN, pool

2

Arquiteturas em PyTorch
LeNet $\rightarrow$ ResNet

3

Treinamento técnico
aug, sched, TB, checkpoint

4

Transfer learning + AMP + Grad-CAM

Mensagem final

Você não precisa decorar arquiteturas. Precisa saber montar uma do zero, treinar com qualidade, e adaptar uma pré-treinada para sua tarefa. Esses três eixos cobrem 95% da prática em visão computacional.

Para continuar estudando

- **Goodfellow, Bengio, Courville** – *Deep Learning*, cap. 9 (Convolutional Networks). Referência teórica canon.
- **PyTorch tutorials** – pytorch.org/tutorials.
- **D2L.ai** – *Dive into Deep Learning*, capítulos 6 e 7. Online, com notebooks.
- **CS231n (Stanford)** – a melhor sequência de notas em inglês para fundamentação matemática.
- **papers clássicos**: AlexNet (2012), VGG (2014), GoogLeNet (2014), ResNet (2015) e EfficientNet (2019). Ler na ordem cronológica.
- **Pacotes úteis**: [timm](#) (centenas de modelos pré-treinados), [torchcam](#) ou [grad-cam](#) (visualizações).

Próxima aula

Aplicaremos estes conceitos em **detecção e segmentação**: Faster R-CNN, Mask R-CNN e YOLOv8 – mesmo backbone CNN que você aprendeu hoje, mas com *outras cabeças* e *outras losses*.

Obrigado!

Dúvidas, sugestões e código no repositório do curso.

Próxima aula: transfer learning aplicado.